

Mattermost Cross Guard

Cross-domain message relay for Mattermost Federal

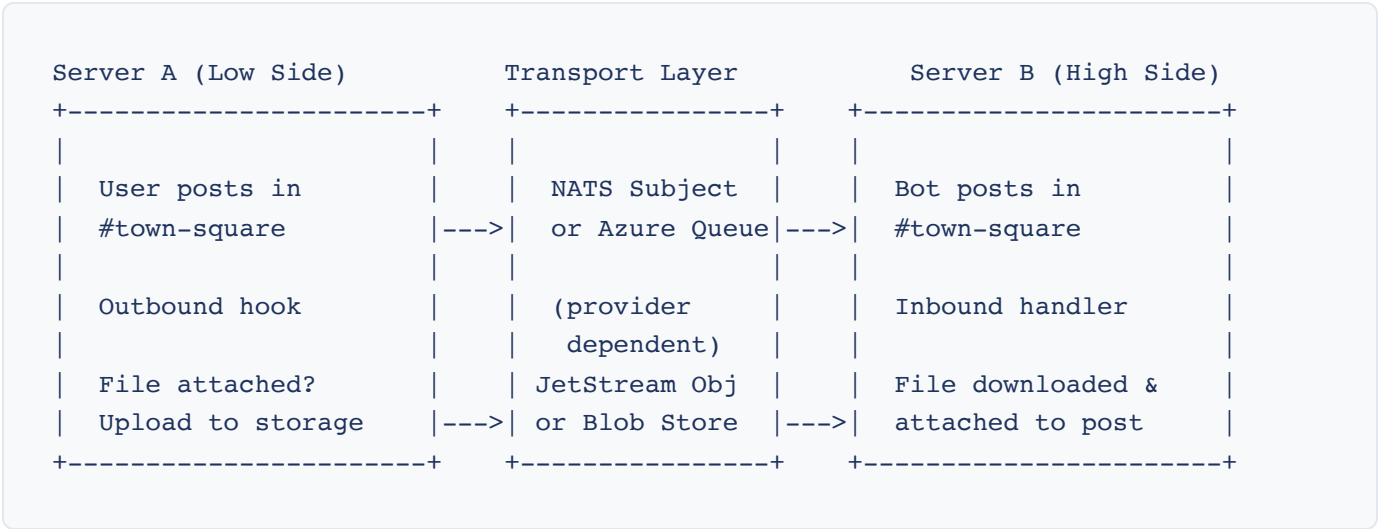
What is Cross Guard?

Cross Guard enables message relay between separate Mattermost servers using NATS or Azure Queue Storage as the transport layer. It is designed for environments where multiple isolated Mattermost instances need to exchange messages across domain boundaries without direct database or API access between servers.

Posts, edits, deletes, reactions, and file attachments flow bidirectionally across server boundaries. Each server maintains its own users, teams, and channels. Cross Guard bridges them at the message level, preserving each server's independence while enabling real-time communication. File attachments are transferred using NATS JetStream Object Store or Azure Blob Storage, depending on the configured provider. File transfer is opt-in per connection.

The plugin integrates with Mattermost's post lifecycle hooks, so relay behavior is transparent to end users. Messages appear in channels as if they were posted locally.

How Federation Works



Two Connection Directions

- **Outbound:** Hooks into Mattermost post lifecycle events (`MessageHasBeenPosted` , `MessageHasBeenUpdated` , `MessageHasBeenDeleted` , reaction events), serializes to a message envelope (JSON or XML, per connection config), and sends via the configured transport.
- **Inbound:** Receives messages from the transport (NATS subscription or Azure queue polling), auto-detects the message format (JSON or XML), deserializes the envelope, and creates, updates, or deletes posts locally.

What Gets Relayed

Event	Relayed?	Notes
New posts	Yes	Including thread replies (root_id preserved)
Post edits	Yes	Updated content replaces original on the remote side
Post deletes	Yes	Matched by original post ID
Reaction adds	Yes	Emoji name and user are relayed. Sync-user reactions are filtered.
Reaction removes	Yes	Matched by emoji name and post ID. Sync-user reactions are filtered.
File attachments	Configurable	Enabled per connection via <code>file_transfer_enabled</code> . Supports allow/deny extension filters.
System messages	No	Filtered out to avoid noise
Relay bot posts	No	Loop prevention via <code>crossguard_relayed</code> property (posts/edits), sync-user filtering (reactions), and delete flag mechanism (deletes)

Key Concepts

Connections

Named connection configurations (NATS or Azure Queue Storage), inbound or outbound, defined in the System Console. Referenced by name in all commands using the format

`direction:name` (e.g., `outbound:relay-to-b`).

Team Initialization

A team must be linked to a connection before any channels in that team can relay messages. This is always the first step. Use `/crossguard init-team` to link a connection.

Channel Initialization

Individual channels within an initialized team are linked to connections. Only linked channels relay messages. Use `/crossguard init-channel` to link a channel.

Inbound Prompts

When an inbound message arrives for a team or channel that hasn't been explicitly accepted, an interactive prompt (Accept/Block buttons) is posted. Team-level prompts appear in town-square; channel-level prompts appear in the target channel. Admins must accept before messages flow. Use `/crossguard reset-prompt` to clear a blocked prompt.

Team Name Rewriting

Maps remote team names to local teams, allowing servers with different team names to relay correctly. Configured per inbound connection using `/crossguard rewrite-team` .

Username Lookup

When enabled (the `UsernameLookup` setting), inbound messages are posted as the matching local user if one exists with the same username. Otherwise, a synthetic relay bot posts them.

Getting Started

1

Configure connections in **System Console > Plugins > Cross Guard**. Add connection details for your chosen provider (NATS or Azure Queue Storage). See the [Admin Configuration Guide](#) for details.

2

Initialize a team: Run `/crossguard init-team [connection-name]` in the team you want to enable.

- 3 **Initialize channels:** Run `/crossguard init-channel [connection-name]` in each channel that should relay messages.
- 4 **Verify:** Run `/crossguard status` to confirm the setup and check that connections are active.

Permission Model

Role	Scope	Notes
SYSTEM ADMIN	All commands, all teams/channels. Can configure connections in System Console. Can test connections via API. Sees global status overview.	Always has full access regardless of <code>RestrictToSystemAdmins</code> setting.
TEAM ADMIN	<code>init-team</code> , <code>teardown-team</code> , <code>init-channel</code> , <code>teardown-channel</code> , <code>reset-prompt</code> , <code>reset-channel-prompt</code> , <code>rewrite-team</code> , <code>status</code> within their team.	Access blocked when <code>RestrictToSystemAdmins</code> is enabled, unless <code>AllowTeamAdminRequests</code> is also enabled (see Request Mode below).
CHANNEL ADMIN	<code>init-channel</code> , <code>teardown-channel</code> within their channel (team must already be initialized).	Access blocked when <code>RestrictToSystemAdmins</code> is enabled.
MEMBER	<code>status</code> (team-scoped view), <code>help</code> .	Can see which connections are active for their team/channel.

Note

When `RestrictToSystemAdmins` is enabled in plugin settings, Team Admins and Channel Admins lose access to connection management commands. Only System Admins can manage connections.

Request Mode

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins regain limited access: `init-team` submits a link request for System Admin approval (instead of linking directly), and `teardown-team` allows direct unlinking without approval.

Channel-level commands remain blocked. System Admins receive request DMs with interactive Approve/Deny buttons. See the [Admin Configuration](#) page for details.

Documentation Pages

Slash Command Reference

Complete documentation for all 9 slash commands, including arguments, permissions, and examples.

Admin Configuration Guide

Connection setup for NATS and Azure, relay settings, example configurations, and troubleshooting.

REST API Reference

All 16 REST endpoints with request/response examples and authentication details.

Error Codes

Every numeric `error_code` emitted in Cross Guard logs, grouped by source file, with descriptions and troubleshooting steps.

Transport Interface

Wire protocol reference: message envelope, payload schemas, connection configuration for NATS and Azure Queue Storage, and complete examples.

Threat Model

STRIDE threat analysis, trust zones, data flow documentation, and CDS deployment recommendations.

CDS Whitepaper

Comprehensive architecture, deployment topologies, security analysis, and cross-domain evaluation guide.

Slash Command Reference

All commands for managing Cross Guard relay connections

Overview

- All commands use the trigger `/crossguard`
- Autocomplete is enabled (hints appear as you type)
- Commands return ephemeral responses (only visible to the user who ran them)
- When multiple connections exist and no name is specified, an interactive selection dialog appears

Quick Reference

Command	Description	Permission	Arguments
<code>init-team [name]</code>	Link connection to team	TEAM ADMIN+	Optional connection name
<code>init-channel [name]</code>	Link connection to channel	CHANNEL ADMIN+	Optional connection name
<code>teardown-team [name]</code>	Unlink connection from team	TEAM ADMIN+	Optional connection name
<code>teardown-channel [name]</code>	Unlink connection from channel	CHANNEL ADMIN+	Optional connection name
<code>reset-prompt <name></code>	Clear team connection prompt	TEAM ADMIN+	Required connection name
<code>reset-channel-prompt <name></code>	Clear channel connection prompt	TEAM ADMIN+	Required connection name

Command	Description	Permission	Arguments
<code>rewrite-team [name] [team]</code>	Set/clear team name rewrite	TEAM ADMIN+	See details
<code>status</code>	Show relay status	ANY MEMBER	None
<code>help</code>	Show help	ANY MEMBER	None

/crossguard init-team [connection-name]

Link a connection to the current team, enabling cross-domain relay for this team.

Permission	Team Admin or System Admin (blocked by <code>RestrictToSystemAdmins</code> ; see Request Mode below)
Arguments	<code>connection-name</code> (optional). The display key of the connection (e.g., <code>outbound:relay-to-b</code>).

Behavior

- If omitted and exactly one connection exists in config, auto-selects it
- If omitted and multiple connections exist, opens an interactive selection dialog with all configured connections
- If specified but not found, returns error listing available connections
- If connection already linked to this team, returns "already linked" message
- On first connection linked to team, adds team to initialized teams list
- Posts announcement in `#town-square` : "Cross Guard connection `outbound:relay-to-b` linked to this team by @username"

Request Mode

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins who run this command submit a **link request** instead of linking directly. All System Admins receive a DM with Approve/Deny buttons. The requester is notified of the outcome. If a request is already pending for the same team and connection, the command returns an error.

Examples

```
/crossguard init-team  
/crossguard init-team outbound:relay-to-b
```

Error Cases

- No connections configured. Add connections in the System Console first.
- Connection not found: xyz. Available connections: outbound:relay-to-b, inbound:relay-from-a
- a request is already pending for this connection (request mode only)

/crossguard init-channel [connection-name]

Link a connection to the current channel. The team must be initialized first with the same connection.

Permission	Channel Admin, Team Admin, or System Admin (blocked by RestrictToSystemAdmins)
Arguments	connection-name (optional, same auto-select/dialog behavior as init-team)

Behavior

- Validates team is initialized first. If the team does not yet have this specific connection linked, the REST API auto-links the team. The slash command requires explicit team initialization via `/crossguard init-team`.
- If connection not linked at team level, returns: `Connection X is not linked to this team. Run /crossguard init-team first.`
- Adds link emoji prefix to channel header (visible indicator)
- Publishes WebSocket event `channel_connections_updated` to update UI indicators
- Posts announcement in the channel

Examples

```
/crossguard init-channel
```

```
/crossguard init-channel outbound:relay-to-b
```

/crossguard teardown-team [connection-name]

Unlink a connection from the current team.

Permission	Team Admin or System Admin (blocked by <code>RestrictToSystemAdmins</code> ; see note below)
-------------------	--

Arguments	<code>connection-name</code> (optional). If omitted and multiple connections linked, opens dialog. If one linked, auto-selects.
------------------	---

Behavior

- If no connections linked: `No connections are linked to this team.`
- If this was the last connection, removes team from initialized teams list
- Posts announcement in `#town-square`
- If a pending connection link request exists for the same team and connection, the request is automatically cancelled and the requester is notified

Request Mode

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can run this command to unlink connections directly (no approval needed for unlinking). This is the only connection management action Team Admins can perform without System Admin approval in request mode.

Warning

This command does NOT automatically teardown channel connections. Channels that still have this connection linked become orphaned. Run `/crossguard teardown-channel` in affected channels first.

Example

```
/crossguard teardown-team outbound:relay-to-b
```

/crossguard teardown-channel [connection-name]

Unlink a connection from the current channel.

Permission	Channel Admin, Team Admin, or System Admin (blocked by <code>RestrictToSystemAdmins</code>)
Arguments	<code>connection-name</code> (optional, same auto-select/dialog behavior)

Behavior

- If no connections linked: `No connections are linked to this channel.`
- If this was the last connection on the channel, removes link emoji prefix from channel header
- Publishes WebSocket event to update UI indicators
- Posts announcement: "All relays for this channel are now inactive."

/crossguard reset-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current team. After clearing, a new prompt will appear on the next inbound message.

Permission	Team Admin or System Admin
Arguments	<code>connection-name</code> (required)

Behavior

- Looks up prompt in KV store for this team + connection name
- If no prompt found: `No prompt found for connection X on this team.`
- Deletes the prompt entry, allowing the system to create a fresh prompt on the next inbound message

Use Case

An admin accidentally blocked an inbound connection and wants to re-evaluate it.

Example

```
/crossguard reset-prompt relay-from-a
```

/crossguard reset-channel-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current channel. Same behavior as `reset-prompt` but operates on channel-level prompts.

Permission	Team Admin or System Admin
------------	----------------------------

Arguments	<code>connection-name</code> (required)
-----------	---

Example

```
/crossguard reset-channel-prompt relay-from-a
```

/crossguard rewrite-team [connection-name] [remote-team-name]

Set or clear a remote team name rewrite for an inbound connection. When set, inbound messages with the specified remote team name are routed to this team instead of requiring a matching local team name.

Permission	Team Admin or System Admin
------------	----------------------------

Arguments	<code>connection-name</code> (required if multiple inbound connections linked, optional if exactly one). <code>remote-team-name</code> (optional; omit to clear the rewrite).
-----------	--

Behavior

- Only works with inbound connections. If no inbound connections linked: `No inbound connections are linked to this team.`
- Sets a rewrite index in the KV store: `connection-name + remote-team-name -> local team ID`

- Returns HTTP 409 (Conflict) if the rewrite index already exists for another team
- Posts audit message in `#town-square` showing who made the change
- When clearing: removes the rewrite index entry and clears `RemoteTeamName` from the team connection metadata

Examples

```

/crossguard rewrite-team relay-from-a remote-team-alpha
/crossguard rewrite-team relay-from-a

```

/crossguard status

Show Cross Guard initialization status for the current team and channel.

Permission	Any team member
Arguments	None

Output Varies by Role

- **Regular users / Team Admins:** Current channel status (relay enabled yes/no), current team status (initialized yes/no), team connections list, channel connections list. Each connection displays file transfer status (e.g., `files off` , `files on` , `files on, allow: .pdf,.docx`).
- **System Admins:** Global overview with all initialized teams (table with display name, team ID, team name, linked connections), all configured connections (table with name, direction, address, auth type, subject, file transfer status), plus current channel status. Warnings shown if connection configuration cannot be parsed.

/crossguard help

Show inline help for all commands. No arguments or special permissions required.

Connection Selection Dialog

When a command needs a connection name and multiple connections are available, an interactive dialog opens automatically.

- Dialog title varies: "Link Connection to Team", "Unlink Connection from Team", etc.
- Dropdown shows all available connections in `direction:name` format
- Dialog submits to `/plugins/crossguard/api/v1/dialog/select-connection`
- State encodes the action and target: `init-team:{teamID}` , `init-channel:{channelID}` , etc.

Admin Configuration Guide

Transport connections, relay settings, and deployment examples

Accessing Plugin Settings

Navigate to **System Console > Plugins > Cross Guard**. The plugin must be enabled before configuration is accessible. Settings are organized into two sections: Connection Configuration and Relay Settings.

Connection Configuration

Understanding Inbound vs Outbound

Direction	Flow	Use When
Outbound	Mattermost → Transport	Your server is the sender . Publishes messages to the configured transport.
Inbound	Transport → Mattermost	Your server is the receiver . Subscribes to or polls messages from the configured transport.

A server can have both inbound and outbound connections simultaneously for bidirectional relay. Multiple connections of the same direction are supported for multi-server topologies.

Provider Selection

Each connection uses either **NATS** or **Azure Queue Storage** as its transport provider. The provider is selected when creating a connection, and the admin console shows only the fields relevant to the chosen provider.

Provider	Transport	File Transfer	Delivery
NATS	Pub/sub messaging via NATS Core	JetStream Object Store	At-most-once
Azure Queue Storage	Polling-based queue (5s interval)	Azure Blob Storage	At-least-once
Azure Blob Storage	Write-ahead log batched to blobs (periodic flush)	Deferred file blobs alongside batch WAL blobs	At-least-once

Common Connection Fields

Field	JSON Key	Required	Validation	Description
Name	name	Yes	Lowercase letters, numbers, and hyphens only. Pattern: <code>^[a-z0-9]+(-[a-z0-9]+)*\$</code> . Must be unique across ALL connections.	Human-readable identifier. Used in all commands and API calls. Examples: <code>relay-to-b</code> , <code>high-side</code> , <code>nats-prod</code>
Provider	provider	Yes	"nats" , "azure-queue" , or "azure-blob" . Empty treated as "nats" .	Transport provider for this connection. Determines which provider-specific fields are required. Default: "nats" .

Field	JSON Key	Required	Validation	Description
File Transfer Enabled	<code>file_transfer_enabled</code>	No	Boolean	Enable file attachment relay for this connection. Default: <code>false</code> .
File Filter Mode	<code>file_filter_mode</code>	No	<code>""</code> , <code>"allow"</code> , or <code>"deny"</code>	Controls whether file extensions are filtered. <code>allow</code> = only listed extensions are relayed. <code>deny</code> = listed extensions are blocked. Empty = all files pass (default).
File Filter Types	<code>file_filter_types</code>	Conditional	Comma-separated extensions. Case-insensitive, leading dot optional.	List of file extensions used by the filter (e.g. <code>.pdf</code> , <code>.docx</code> , <code>.png</code>). Required when <code>file_filter_mode</code> is <code>allow</code> or <code>deny</code> .
Message Format	<code>message_format</code>	No	<code>"json"</code> or <code>"xml"</code> . Empty treated as <code>"json"</code> .	Wire format for outbound messages. Use <code>"xml"</code> when sending through a Cross Domain Solution (CDS) that requires XML. Inbound messages are auto-detected regardless of this setting. Only applies to outbound connections. Default: <code>"json"</code> .

NATS Provider Fields

These fields appear when the provider is set to **NATS**. In JSON, they are nested under the `"nats"` key (e.g., `"nats": {"address": "...", "subject": "..."}`).

Field	JSON Key	Required	Validation	Description
Address	<code>address</code>	Yes	Non-empty string	NATS server URL. Format: <code>nats://hostname:port</code> . Default port is 4222. Examples: <code>nats://nats.example.com:4222</code> <code>nats://10.0.1.50:4222</code>
Subject	<code>subject</code>	Yes	Must start with <code>crossguard.</code> prefix	NATS subject to publish/subscribe on. Both sides MUST use the same subject. Examples: <code>crossguard.relay.messages</code> , <code>crossguard.prod.sync</code>
TLS Enabled	<code>tls_enabled</code>	No	Boolean	Enable TLS encryption. Required in production with TLS-enabled NAT servers.
Auth Type	<code>auth_type</code>	No	<code>none</code> , <code>token</code> , or <code>credentials</code> . Empty treated as <code>none</code> .	Authentication method for the NATS server.
Token	<code>token</code>	Conditional	Required when <code>auth_type</code> is <code>token</code>	Authentication token for token-based auth.
Username	<code>username</code>	Conditional	Required when <code>auth_type</code> is <code>credentials</code>	Username for credential-based auth.
Password	<code>password</code>	Conditional	Required when <code>auth_type</code> is <code>credentials</code>	Password for credential-based authentication.
Client Certificate	<code>client_cert</code>	No	If provided, <code>client_key</code>	Path to client TLS certificate file (requires <code>tls_enabled</code>).

Field	JSON Key	Required	Validation	Description
			must also be provided	
Client Key	<code>client_key</code>	No	If provided, <code>client_cert</code> must also be provided	Path to client TLS private key file.
CA Certificate	<code>ca_cert</code>	No	-	Path to CA certificate for verifying the NATS server's TLS certificate.

Azure Queue Provider Fields

These fields appear when the provider is set to **Azure Queue Storage**. In JSON, they are nested under the `"azure_queue"` key (e.g., `"azure_queue": {"queue_service_url": "...", "account_name": "...", "account_key": "...", "queue_name": "..."}`).

Field	JSON Key	Required	Validation	Description
Queue Service URL	<code>queue_service_url</code>	Yes	Non-empty, valid URL	Azure Queue Storage URL. For Azure, the standard format is <code>https://<account_name>.queue.azure.com/</code> . For Azurite (local emulator), the format is <code>http://azurite:10000/</code> .
Blob Service URL	<code>blob_service_url</code>	Conditional	Required when <code>file_transfer_enabled</code> is <code>true</code> . Must be a valid URL.	Azure Blob Storage URL. For Azure, the standard format is <code>https://<account_name>.blob.azure.com/</code> .
Account Name	<code>account_name</code>	Yes	Non-empty string	Azure Storage account name.
Account Key	<code>account_key</code>	Yes	Non-empty string	Azure Storage account key (base64). The key is displayed in the Azure Portal under the account's keys.
Queue Name	<code>queue_name</code>	Yes	Non-empty string	Name of the queue. Required if using the queue if it's not the default queue.

Field	JSON Key	Required	Validation	Description
				the same que
Blob Container Name	<code>blob_container_name</code>	Conditional	Required when <code>file_transfer_enabled</code> is <code>true</code>	Name of the , Auto-created use the same

Azure Blob Provider Fields

These fields appear when the provider is set to **Azure Blob Storage** (the batched WAL transport, distinct from the Azure Queue provider above). In JSON, they are nested under the `"azure_blob"` key. Instead of a queue, the outbound side batches messages into a write-ahead log blob and flushes it on an interval; the inbound side polls the container, takes a short-lived lease on each batch blob, processes it, and deletes it. File attachments are stored as deferred blobs alongside the batch blobs in the same container.

Field	JSON Key	Required	Validation	Description
Service URL	<code>service_url</code>	Yes	Non-empty, valid URL	Azure Blob Storage servi Azure, the standard form <code>https://<account>.bl</code> For Azurite, use <code>http://azurite:10000</code>
Account Name	<code>account_name</code>	Yes	Non-empty string	Azure Storage account n. <code>account_key</code> to form a
Account Key	<code>account_key</code>	Yes	Non-empty string	Azure Storage account sl (base64). Treat as a secr
Blob Container Name	<code>blob_container_name</code>	Yes	Non-empty string	Name of the Azure Blob c batch WAL blobs and (wt enabled) deferred file blc does not exist. Both side: container name.

Field	JSON Key	Required	Validation	Description
Flush Interval	<code>flush_interval_seconds</code>	No	Integer. If set, must be at least <code>5</code> .	How often the outbound write-ahead log blob and seconds. Lower values reduce latency at the cost of more
Blob Lock Max Age	<code>blob_lock_max_age_seconds</code>	No	Integer. If set, must be at least <code>30</code> .	Maximum age of a blob lock side considers it stale and Default: <code>300</code> seconds (5 minutes) crashed inbound node from

Validation Rules Summary

- Names must be globally unique across inbound AND outbound connections
- Names must be lowercase alphanumeric with hyphens (no spaces, underscores, uppercase)
- Subject must always start with `crossguard.`
- Both sides of a relay must use the same NATS subject
- Auth credentials must be complete (both username AND password for credentials auth)
- TLS cert and key must be provided as a pair (enforced at config save time)
- Azure Queue connections require `queue_service_url`, `account_name`, `account_key`, and `queue_name`
- Azure Queue connections require `blob_service_url` and `blob_container_name` when file transfer is enabled
- Azure Blob connections require `service_url`, `account_name`, `account_key`, and `blob_container_name`
- Azure Blob `flush_interval_seconds` must be at least 5 when set; `blob_lock_max_age_seconds` must be at least 30 when set

Relay Settings

Username Lookup

Setting	UsernameLookup
Type	Boolean
Default	true

When enabled: Inbound messages look for a local user with the same username as the remote sender. If found, the message is posted as that local user. This creates a seamless experience where messages appear to come from the correct person.

When disabled: All inbound messages are posted by the Cross Guard relay bot, with the remote username shown in the message formatting. Use this if usernames don't match across servers.

Restrict to System Admins

Setting	RestrictToSystemAdmins
Type	Boolean
Default	false

When enabled: Only System Admins can run `init-team`, `teardown-team`, `init-channel`, `teardown-channel`, `reset-prompt`, `reset-channel-prompt`, and `rewrite-team`. Team Admins and Channel Admins are denied access.

When disabled: Team Admins can manage team connections, Channel Admins can manage channel connections (the default permission hierarchy).

Tip

Enable this setting in high-security environments where only centralized admins should control cross-domain relay. Pair it with `AllowTeamAdminRequests` below to let Team Admins request connections without granting them direct control.

Allow Team Admin Requests

Setting	AllowTeamAdminRequests
---------	------------------------

Type	Boolean
Default	false
Requires	RestrictToSystemAdmins must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Team Admins can submit connection link requests via `/crossguard init-team` or the REST API. The request is sent as a direct message to every System Admin with interactive **Approve** and **Deny** buttons. Team Admins can also unlink connections directly with `/crossguard teardown-team` (no approval needed for unlinking).

When disabled: Team Admins are fully blocked from connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Team Admins already have direct access.

Request workflow

When a Team Admin submits a link request, all System Admins receive a DM from the Cross Guard bot with the requester's name, team, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

File Transfer Settings

File transfer is configured per connection, not globally. Each connection can independently enable or disable file attachment relay and apply extension filters.

Requirements

NATS Provider

- **JetStream-enabled NATS server:** File transfer uses NATS JetStream Object Store. Core NATS alone is not sufficient. The NATS server must have JetStream enabled.
- **Object Store bucket:** The plugin auto-creates a `crossguard-files` bucket with a 1-hour TTL on first use. No manual NATS configuration is needed.

- **Max file size:** Governed by the Mattermost server's `FileSettings.MaxFileSize` configuration (default 100 MB). Files exceeding this limit are silently skipped with a log warning.

Azure Provider

- **Blob container:** File transfer uses Azure Blob Storage. The plugin auto-creates the container specified in `blob_container_name` if it does not exist.
- **Blob metadata:** Headers (post ID, connection name, filename) are stored as Base64-encoded JSON in the blob's `crossguard_headers` metadata key.
- **Polling interval:** The inbound blob watcher polls every 15 seconds. Files are deleted after successful processing.
- **Max file size:** Same as NATS: governed by the Mattermost server's `FileSettings.MaxFileSize` configuration.

Filter Examples

Goal	file_filter_mode	file_filter_types
Allow all files (default)	<code>""</code>	<code>""</code>
Allow only PDFs and images	<code>"allow"</code>	<code>".pdf,.png,.jpg,.gif"</code>
Block executables	<code>"deny"</code>	<code>".exe,.bat,.sh,.cmd"</code>

Note

For bidirectional file relay, both the outbound and inbound connections must have `file_transfer_enabled` set to `true`. File filters are evaluated independently on each side, so the outbound connection can filter what it sends and the inbound connection can filter what it accepts.

Testing Connections

The admin console UI includes a "Test Connection" button for each configured connection.

- **API endpoint:** `POST /plugins/crossguard/api/v1/test-connection` (System Admin only)

- **NATS outbound test:** Publishes a message with a unique ID and confirms flush
- **NATS inbound test:** Subscribes to the subject and confirms the subscription is active
- **Azure test:** Validates the connection string by enqueueing and deleting a test message
- Tests use a temporary connection (does not affect the running plugin's connections)
- The `direction` query parameter is NATS-specific; Azure tests validate connectivity regardless of direction

Example Configurations

Dual-Server Setup (Server A sends to Server B)

Server A (sender)

```
Outbound Connections:
[
  {
    "name": "relay-to-b",
    "provider": "nats",
    "nats": {
      "address": "nats://nats.example.com:4222",
      "subject": "crossguard.relay.messages",
      "auth_type": "token",
      "token": "shared-secret-token"
    },
    "file_transfer_enabled": true,
    "file_filter_mode": "allow",
    "file_filter_types": ".pdf,.docx,.png"
  }
]

Inbound Connections: []
```

Server B (receiver)

```
Inbound Connections:
[
  {
    "name": "relay-from-a",
    "provider": "nats",
    "nats": {
      "address": "nats://nats.example.com:4222",
      "subject": "crossguard.relay.messages",

```

```
"auth_type": "token",
"token": "shared-secret-token"
},
"file_transfer_enabled": true
}]
```

```
Outbound Connections: []
```

Bidirectional Setup (Both servers send and receive)

Server A

```
Outbound: [{"name": "to-b", "provider": "nats", "nats": {"address": "nats://nats:4222"}},
Inbound:  [{"name": "from-b", "provider": "nats", "nats": {"address": "nats://nats:4222"}},
```

Server B

```
Outbound: [{"name": "to-a", "provider": "nats", "nats": {"address": "nats://nats:4222"}},
Inbound:  [{"name": "from-a", "provider": "nats", "nats": {"address": "nats://nats:4222"}},
```

Note

Each direction uses a different subject to avoid echo loops. Server A publishes on `crossguard.a-to-b` and Server B subscribes to it, and vice versa.

Docker Dev Environment

- **Server A:** `http://low.test:8075` (admin/password, usera/password, Team: Test A)
- **Server B:** `http://high.test:8076` (admin/password, userb/password, Team: Test B)
- **NATS:** `nats://nats:4222` (from plugin's perspective inside Docker network)
- **Setup:** `make docker-setup`, **Deploy:** `make deploy`

Azure Queue Storage Setup

Server A (sender via Azure Queue)

Outbound Connections:

```
[{
  "name": "azure-to-b",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  },
  "file_transfer_enabled": true
}]
```

Server B (receiver via Azure Queue)

Inbound Connections:

```
[{
  "name": "azure-from-a",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  },
  "file_transfer_enabled": true
}]
```

Azure Blob variant (batched WAL transport)

Outbound Connections:

```
[{
  "name": "azure-blob-to-b",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal",
    "flush_interval_seconds": 60,
  }
}]
```

```
"blob_lock_max_age_seconds": 300
},
"file_transfer_enabled": true
}]
```

Note

Both sides share the same Azure Storage account, queue name, and blob container. The outbound side enqueues messages and uploads blobs; the inbound side polls and processes them.

Troubleshooting

Problem	Solution
"No connections configured"	Add connections in System Console > Plugins > Cross Guard first.
NATS connection test fails	Verify NATS server is reachable, check address/port, verify auth credentials match.
Messages not relaying	Run <code>/crossguard status</code> . Verify both team AND channel are initialized. Check that transport identifiers match on both sides (NATS subject for NATS connections, queue name for Azure connections).
"Connection not found"	Connection name is case-sensitive and uses <code>direction:name</code> format (e.g., <code>outbound:relay-to-b</code>).
Orphaned connections	If a connection name is changed in config, existing team/channel links reference the old name. Teardown and re-init with the new name.
Inbound messages blocked	Check if an inbound prompt was blocked. Run <code>/crossguard reset-prompt <name></code> to clear it and re-evaluate.
Wrong team receives messages	Check team name rewrite settings with <code>/crossguard status</code> . Remote team names must match or be rewritten.

Problem	Solution
Files not transferring	Verify <code>file_transfer_enabled</code> is <code>true</code> on both outbound and inbound connections. For NATS, ensure the server has JetStream enabled (not just core NATS). For Azure, verify <code>blob_container_name</code> is set on both outbound and inbound connections.
"File semaphore full" log warnings	Too many concurrent file transfers. The limit is 32 simultaneous file operations. Large files or slow connections can cause congestion. Consider reducing file sizes or upgrading network bandwidth.
File blocked by filter	Check <code>file_filter_mode</code> and <code>file_filter_types</code> on the connection. An <code>allow</code> filter rejects extensions not in the list; a <code>deny</code> filter rejects extensions in the list.
File appears after text message	Normal behavior. Files arrive asynchronously via the file watcher (NATS Object Store watcher or Azure Blob poller). The inbound watcher retries post mapping lookup up to 3 times with backoff to wait for the text message to be processed first.
Large files silently dropped	Files larger than the server's <code>MaxFileSize</code> setting (default 100 MB) are skipped with a log warning. Check <code>FileSettings.MaxFileSize</code> in the Mattermost server configuration.
Azure connection test fails	Verify the connection string is correct. Check that the storage account is accessible from the server. Ensure the account key has not been rotated.
Azure messages delayed	Azure Queue Storage uses polling (5-second intervals for messages, 15-second intervals for files). This is normal. For lower latency, use the NATS provider.
Azure message too large	Azure Queue Storage has a 48,000-byte message size limit (before Base64 encoding to the 64 KB wire limit). Messages exceeding this limit have their text truncated to fit. Long posts may lose content.
Duplicate messages with Azure	Azure Queue provides at-least-once delivery. The plugin uses idempotency checks (post ID mapping) to prevent duplicates, but brief visibility timeout races can cause rare duplicates during error recovery.
Azure blob files not appearing	Verify <code>blob_container_name</code> is set on both sides. Check that <code>file_transfer_enabled</code> is <code>true</code> on both outbound and inbound connections. The blob watcher polls every 15 seconds.

REST API Reference

All Cross Guard REST endpoints with request and response examples

Overview

Base Path	/plugins/crossguard/api/v1/
Authentication	All endpoints require the Mattermost-User-Id header (set automatically by Mattermost for authenticated requests).
Request Limit	5 MB max request body
Response Format	JSON
Error Format	{"error": "message"} with appropriate HTTP status code

Connection Testing

POST

/api/v1/test-connection

SYSTEM ADMIN

Test a connection configuration before saving. Supports NATS, Azure Queue, and Azure Blob providers.

Query Parameters

direction	Optional. inbound or outbound (defaults to outbound). NATS-specific; ignored for Azure Queue and Azure Blob connections.
-----------	---

Request Body (NATS)

```
{
  "name": "relay-to-b",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
```

```
"subject": "crossguard.relay.messages",
"tls_enabled": false,
"auth_type": "token",
"token": "my-secret-token"
}
}
```

Request Body (Azure Queue)

```
{
  "name": "azure-relay",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay"
  }
}
```

Request Body (Azure Blob)

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal"
  }
}
```

Validation

NATS

- `nats.address` required
- `nats.subject` required and must start with `crossguard.`
- `nats.auth_type` must be `none`, `token`, `credentials`, or empty
- `nats.token` required if `auth_type` is `token`
- `nats.username` + `nats.password` required if `auth_type` is `credentials`

Azure Queue

- `azure_queue.queue_service_url` required
- `azure_queue.account_name` required
- `azure_queue.account_key` required
- `azure_queue.queue_name` required

Azure Blob

- `azure_blob.service_url` required
- `azure_blob.account_name` required
- `azure_blob.account_key` required
- `azure_blob.blob_container_name` required

Success Response (outbound) - 200

```
{"status": "ok", "message": "Test message sent with ID: abc123", "id": "abc123"}
```

Success Response (inbound) - 200

```
{"status": "ok", "message": "Connected and subscribed to subject successfully"}
```

Success Response (Azure Queue) - 200

```
{"status": "ok", "message": "Azure Queue Storage connection test successful"}
```

Success Response (Azure Blob) - 200

```
{"status": "ok", "message": "Azure Blob Storage connection test successful"}
```

Error Responses

Status	Meaning
400	Validation error
401	Not authenticated

Status	Meaning
403	Not system admin
502	Transport connection failed (NATS unreachable, or Azure credentials/service URL invalid)

Team Management

POST `/api/v1/teams/{team_id}/init` **TEAM ADMIN+**

Link a connection to a team (API equivalent of `/crossguard init-team`).

Path Parameters

team_id 26-character Mattermost team ID

Query Parameters

connection_name Optional. Connection in `direction:name` format (e.g., `outbound:relay-to-b`).

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

Request Mode Response - 200

When the caller is a Team Admin and both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, the endpoint submits a link request instead of linking directly:

```
{
  "status": "request_submitted",
  "message": "Your request to link connection `outbound:relay-to-b` ...",
  "team_id": "abc123...",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
400	Invalid team_id, no connections configured, or connection not found (includes <code>connections</code> array of available names)
403	Insufficient permissions
404	Team not found
409	A request is already pending for this connection (request mode only)
500	Internal error

POST `/api/v1/teams/{team_id}/teardown` **TEAM ADMIN+**

Unlink a connection from a team. If a pending connection link request exists for the same team and connection, it is automatically cancelled.

Path Parameters

<code>team_id</code>	26-character Mattermost team ID
----------------------	---------------------------------

Query Parameters

<code>connection_name</code>	Optional. Connection to unlink.
------------------------------	---------------------------------

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

POST `/api/v1/channels/{channel_id}/init` **CHANNEL ADMIN+**

Link a connection to a channel (team auto-inits if needed).

Path Parameters

channel_id 26-character Mattermost channel ID

Query Parameters

connection_name Optional. Connection in **direction:name** format.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
400	Invalid input or no connections
403	Insufficient permissions

Status	Meaning
--------	---------

404	Channel not found
-----	-------------------

500	Internal error
-----	----------------

POST `/api/v1/channels/{channel_id}/teardown`

CHANNEL ADMIN+

Unlink a connection from a channel.

Path Parameters

<code>channel_id</code>	26-character Mattermost channel ID
-------------------------	------------------------------------

Query Parameters

<code>connection_name</code>	Optional. Connection to unlink.
------------------------------	---------------------------------

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Status

GET `/api/v1/status`

SYSTEM ADMIN

Global status of all initialized teams and transport connections.

Response - 200

```
{
  "teams": [
    {
      "team_id": "abc123...",
      "team_name": "test-a",
      "display_name": "Test A",
      "linked_connections": [
        {"direction": "outbound", "connection": "relay-to-b"}
      ]
    }
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "provider": "nats",
      "address": "nats://nats:4222",
      "auth_type": "token",
      "subject": "crossguard.relay.messages",
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    },
    {
      "name": "azure-relay",
      "direction": "outbound",
      "provider": "azure-queue",
      "queue_name": "crossguard-relay",
      "file_transfer_enabled": false,
      "message_format": "json"
    },
    {
      "name": "azure-blob-relay",
      "direction": "outbound",
      "provider": "azure-blob",
      "blob_container_name": "crossguard-wal",
      "file_transfer_enabled": false,
      "message_format": "json"
    }
  ]
}
```

Connection details are redacted. Tokens, passwords, and TLS cert paths are not included.

GET

/api/v1/teams/{team_id}/status

TEAM MEMBER

Initialization and connection status for a specific team.

Response - 200

```
{
  "team_id": "abc123...",
  "team_name": "test-a",
  "team_display_name": "Test A",
  "initialized": true,
  "request_mode": false,
  "linked_connections": [
    {"direction": "outbound", "connection": "relay-to-b"}
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Response Fields

orphaned

true when the connection is linked to the team but no longer exists in the plugin configuration.

remote_team_name

Set when a team name rewrite is configured for this inbound connection.

request_mode

`true` when the calling user is a Team Admin and the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled). Omitted for System Admins.

request_pending

`true` when a link request is pending for this unlinked connection. Only populated when the plugin is in request mode.

GET**/api/v1/channels/{channel_id}/status****CHANNEL MEMBER**

Connection status for a specific channel.

Response - 200

```
{
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "channel_display_name": "Town Square",
  "team_name": "Test A",
  "team_connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Note

Returns an error for DM and group message channels: "Cross Guard is not available for direct or group messages"

GET**/api/v1/channels/connections****AUTHENTICATED**

Bulk query which channels have active Cross Guard connections.

Query Parameters

ids

Required. Comma-separated channel IDs (max 2048 IDs).

Response - 200

```
{
  "channel_id_1": "outbound:relay-to-b,inbound:relay-from-a",
  "channel_id_2": "outbound:relay-to-b"
}
```

- Only returns channels the user is a member of
- Invalid IDs and channels without connections are omitted
- Used internally by the webapp to show channel indicators

Team Name Rewriting

POST**/api/v1/teams/{team_id}/rewrite****TEAM ADMIN+**

Set a remote team name rewrite for an inbound connection.

Request Body

```
{
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Validation

- Both `connection` and `remote_team_name` must be non-empty
- Connection must be inbound and linked to this team

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Error Responses

Status	Meaning
400	Validation error, connection not inbound, or not linked
403	Insufficient permissions
409	Rewrite index already exists for this connection + remote_team_name pair on another team
500	Internal error

DELETE `/api/v1/teams/{team_id}/rewrite` **TEAM ADMIN+**

Clear a remote team name rewrite.

Query Parameters

`connection` Required. Connection name to clear the rewrite for.

Success Response - 200

```
{
  "status": "ok",
```

```
"team_id": "abc123...",  
"connection": "relay-from-a"  
}
```

Connection Prompts

Note

These endpoints are called by interactive buttons in prompt messages. They are not intended to be called directly.

POST `/api/v1/prompt/accept` **TEAM ADMIN+**

Accept button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."} 
```

Links inbound connection to team, deletes the prompt, and updates the prompt post message to show acceptance. Returns a `PostActionIntegrationResponse` .

POST `/api/v1/prompt/block` **TEAM ADMIN+**

Block button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."} 
```

Sets prompt state to `blocked` and updates the prompt post message. Connection stays unlinked until `reset-prompt` is used.

POST `/api/v1/prompt/channel/accept` **TEAM ADMIN+**

Accept button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."} 
```

Links inbound connection to channel, deletes prompt, and updates the post.

POST `/api/v1/prompt/channel/block` **TEAM ADMIN+**

Block button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."} 
```

Sets channel prompt state to `blocked` and updates the post.

Connection Requests

Note

These endpoints are called by interactive buttons in connection request DMs sent to System Admins. They are not intended to be called directly. Requests are created when a Team Admin runs

`/crossguard init-team` (or the REST equivalent) while the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled).

POST `/api/v1/request/approve` **SYSTEM ADMIN**

Approve a pending connection link request. Links the connection to the team, deletes the request, updates all admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."} 
```

Behavior

- Executes the team init (same as a direct System Admin link)
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse` .

POST `/api/v1/request/deny` **SYSTEM ADMIN**

Deny a pending connection link request. Opens an interactive dialog for the admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."} 
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/request/deny-submit`

Returns a `PostActionIntegrationResponse` .

POST

`/api/v1/request/deny-submit`

SYSTEM ADMIN

Dialog submission handler for deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state : team_id|conn_key`
- `submission.reason` : optional denial reason text

Behavior

- Deletes the pending request
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Dialog Handling

POST

`/api/v1/dialog/select-connection`

AUTHENTICATED

Interactive dialog handler when user selects a connection from the dropdown.

Request Body

`SubmitDialogRequest` with:

- `connection_name` field: selected connection in `direction:name` format
- `state` field: `action:target_id` (e.g., `init-team:abc123` , `teardown-channel:xyz789`)

Supported Actions

Action	Description
<code>init-team</code>	Link connection to team
<code>teardown-team</code>	Unlink connection from team
<code>init-channel</code>	Link connection to channel
<code>teardown-channel</code>	Unlink connection from channel

Validates permissions, performs the action, and sends an ephemeral message with the result. Returns `200` on success.

Message Transport Interface

Cross Guard supports NATS and Azure Queue Storage as transport layers for relaying messages between Mattermost instances. For the complete wire protocol reference, including all message envelope schemas, payload definitions, connection configuration, data flow diagrams, and full wire format examples, see the dedicated [Transport Interface Reference](#).

Error Codes

Reference for every numeric `error_code` emitted by Cross Guard logs

How to Read This Page

Every `p.API.Log*` call in the Cross Guard plugin (non-test code) carries a stable numeric `error_code` as its first key-value pair. Operators can grep production logs for the integer value even if the Go constant name changes later. Numbers are never reused or renumbered once assigned.

Example log line:

```
{"level":"error","msg":"Failed to check channel connections","error_code":10000,"chan
```

Codes are allocated per source file in non-overlapping 1000-wide ranges. The first two digits identify the subsystem. Each row below lists the exact log message, the fields you can expect alongside `error_code` , and the level at which the message is emitted (`DEBUG` , `INFO` , `WARN` , `ERROR`).

Informational and warning codes

Not every code represents a failure. Some mark successful events (connection established, subscription started, retry succeeded) or expected drops (policy filters, max retries reached). Check the log level first: `INFO` lines never require action, `WARN` lines are usually recoverable, and `ERROR` lines indicate a broken operation.

Range Quick Reference

Range	Source file	Subsystem	Count
10000–10999	hooks.go	Mattermost post and reaction event hooks	12
11000–11999	api.go	REST API handlers	17

Range	Source file	Subsystem	Count
12000–12999	configuration.go	Plugin configuration lifecycle	2
13000–13999	command.go	Slash command handlers	1
14000–14999	service.go	Team and channel initialization service	34
15000–15999	inbound.go	Inbound message dispatch and file watcher	46
16000–16999	connections.go	Outbound publisher and file upload	14
17000–17999	sync_user.go	Synthetic relay-user resolution	4
18000–18999	azure_blob_provider.go	Azure Blob WAL provider	49
19000–19999	azure_provider.go	Azure Queue provider	10
20000–20999	nats_provider.go	NATS transport provider	4
21000–21999	prompt.go	Inbound connection prompts	17
22000–22999	retry_dispatch.go	Missing-parent post retry queue	5
23000–23999	store/caching.go	Cluster cache invalidation	1
24000–24999	request.go	Team admin connection request workflow	15

hooks.go (10000–10999)

Emitted from Mattermost post and reaction lifecycle hooks (`MessageHasBeenPosted` , `MessageHasBeenUpdated` , `MessageHasBeenDeleted` , `ReactionHasBeenAdded` , `ReactionHasBeenRemoved`). These run on every message event on the server, so transient failures here can prevent relay of individual messages.

Code	Name / Level	Log message and context	Troubleshooting
10000	<code>HooksChannelConnCheckFailed</code> ERROR	"Failed to check channel connections" with <code>channel_id</code> . The post	Check the <code>error</code> field for the underlying KV stor

Code	Name / Level	Log message and context	Troubleshooting
		hook tried to look up which Cross Guard connections are linked to the channel where a message was just posted, and the KV store lookup failed.	error. Usually transient (store restart, read pressure). If repeated for the same <code>channel_id</code> run <code>/crossguard status</code> in that channel to compare what the store returns. If the store is corrupt, remove and re-add the channel link with <code>teardown-channel</code> then <code>init-channel</code> .
10001	HooksGetChannelFailed ERROR	"Failed to get channel for relay" with <code>channel_id</code> . Mattermost returned an error when the hook asked for the channel record by ID.	Channel may have been deleted between the event and the hook firing which is safe to ignore. If persistent check Mattermost database health and that the plugin has read-channel permission. Grep for <code>channel_id</code> to see whether one specific channel is affected.
10002	HooksTeamConnCheckFailed ERROR	"Failed to check team connections" with <code>team_id</code> . The hook tried to read the team-level connection list before relaying a message.	Same pattern as 10000: inspect the adjacent <code>error</code> field. The team may have a partial or corrupt link. Run <code>/crossguard</code>

Code	Name / Level	Log message and context	Troubleshooting
			<code>status</code> in the tea confirms whether the link is readable
10003	HooksGetTeamFailed ERROR	"Failed to get team for relay" with <code>team_id</code> . Mattermost returned an error when the hook asked for the team record.	Team may have be deleted, or Mattermost is und load. Check database health. Transient single occurrences are s to ignore.
10004	HooksRelaySemaphoreFull WARN	"Relay semaphore full, dropping event" with <code>context</code> . The per-process relay concurrency semaphore was saturated when a new event arrived, so the message was dropped rather than blocking the hook.	Raise <code>RelayConcurren</code> in System Console Plugins > Cross Guard. Look for correlated slowne in <code>connections</code> . logs (16001, 1600 16007): the downstream provi is likely slow or unreachable. Sustained semaphore-full events mean the plugin cannot kee up with the local message rate.
10005	HooksGetUserForPostFailed ERROR	"Failed to get user for relay" with <code>user_id</code> . The create-post hook failed to load the author before serializing the envelope.	User may have be deleted, or the plugin lacks read-user permission. Check the <code>error</code> field. Occasional failures are safe to ignore because th next message for

Code	Name / Level	Log message and context	Troubleshooting
			the same user will retry.
10006	HooksGetUserForUpdateFailed ERROR	"Failed to get user for relay" with <code>user_id</code> . Same as 10005, but from the edit-post hook.	Same as 10005.
10007	HooksDeleteFlagCheckFailed ERROR	"Failed to check delete flag, skipping relay to avoid loop" with <code>post_id</code> . The delete hook could not check the <code>crossguard_delete_flag</code> used to break relay loops, so it skipped relaying this delete.	Inspect the KV store error. A persistent failure means delete events stop propagating. Check that the KV store is reachable and not out of quota. The flag has a short TTL so transient failures self-heal.
10008	HooksGetPostForReactAddFailed ERROR	"Failed to get post for reaction relay" with <code>post_id</code> . The reaction-add hook could not load the target post.	Post may have been deleted concurrently. Safe to ignore single instances. If repeated, check Mattermost post store health.
10009	HooksGetUserForReactAddFailed ERROR	"Failed to get user for reaction relay" with <code>user_id</code> . Reaction-add hook could not load the reacting user.	Same as 10005.
10010	HooksGetPostForReactRemFailed ERROR	"Failed to get post for reaction relay" with <code>post_id</code> . Reaction-remove hook could not load the target post.	Same as 10008.

Code	Name / Level	Log message and context	Troubleshooting
10011	HooksGetUserForReactRemFailed ERROR	"Failed to get user for reaction relay" with <code>user_id</code> . Reaction-remove hook could not load the user whose reaction was removed.	Same as 10005.

api.go (11000-11999)

Emitted from REST API handlers. Most codes correspond to a System Console action (test connection, set rewrite) or to a slash-command action dispatched via the API.

Code	Name / Level	Log message and context	Troubleshooting
11000	APINATSTestConnectFailed ERROR	"NATS connection test failed" with <code>address</code> . The <code>/test-connection</code> handler could not dial the NATS broker with the supplied configuration.	Verify network reachability settings, and auth credentials (username/password) in the plugin host, try <code>telnet <host> <port></code> or <code>openssl s_client -connect <host>:<port></code> brokers.
11001	APIBuildTestMessageFailed ERROR	"Failed to build test message". The test-connection path could not construct the test envelope.	Internal bug. Capture the test message field and file an issue. unless the plugin is up
11002	APIPublishTestMessageFailed ERROR	"Failed to publish test message" with <code>subject</code> . Dialing succeeded but the publish call failed.	Verify that the NATS has permission on the connection. Check that the subject starts with a slash. Check broker-side log operations.

Code	Name / Level	Log message and context	Troubleshooting
11003	APIFlushTestMessageFailed ERROR	"Failed to flush test message" with <code>subject</code> . Publish was buffered but flush (forcing delivery) failed, usually because the connection dropped.	Retry the test. If repro closing connections sl Check TLS hostname , keepalive settings, and firewalls.
11004	APITestMessageSent INFO	"Test message sent" with <code>subject</code> , <code>msg_id</code> . Informational confirmation that the outbound test envelope was published.	Informational. The <code>ms</code> correlated with <code>InboundTestMessage</code> (15011) on the receiver
11005	APISubscribeInboundTestFailed ERROR	"Failed to subscribe for inbound test" with <code>subject</code> . The inbound test-connection path could not subscribe to the NATS subject.	Verify the account has on the subject. Multi-t are often scoped to sp
11006	APIFlushNATSConnFailed ERROR	"Failed to flush NATS connection". Cleanup flush failed on the short-lived test client.	Non-fatal if the primar success. Indicates the early.
11007	APIInboundTestSubscribeOK INFO	"Inbound test subscription successful" with <code>subject</code> . Informational.	Informational.

Code	Name / Level	Log message and context	Troubleshooting
11008	APIAzureQueueTestFailed ERROR	"Azure Queue connection test failed". The Azure Queue test could not reach or authenticate against the configured queue endpoint.	Check <code>queue_service_account_name</code> , <code>account_name</code> , <code>queue_name</code> . For Azure emulator is running an Azurite format (<code>http://host:10001</code>). For production, confirm the account key has no
11009	APIAzureBlobTestFailed ERROR	"Azure Blob connection test failed". The Azure Blob test could not reach or authenticate against the configured blob container.	Check <code>service_url</code> , <code>blob_container_name</code> . If the container already exists, credentials can create the blob emulator is running (default 10000). See A the full Azure setup.
11010	APIGetUserFailed ERROR	"Failed to get user" with <code>user_id</code> . An API handler could not load the requesting user from Mattermost.	Usually means an expired transient store error. If the webapp. If repeated check Mattermost store
11011	APIInitChannelGetTeamConns ERROR	"Failed to get team connections" with <code>team_id</code> . Channel-init endpoint could not load the owning team's connections (needed to auto-init the team).	Confirm the team exists <code>/crossguard init-team</code> . Inspect the KV store e
11012	APITeardownChannelGetChanConns ERROR	"Failed to get channel connections" with <code>channel_id</code> . Channel-teardown endpoint could not	Inspect the KV store e

Code	Name / Level	Log message and context	Troubleshooting
		load the channel's current connections.	
11013	APITeardownTeamGetTeamConns ERROR	"Failed to get team connections" with <code>team_id</code> . Team-teardown endpoint could not load the team's connections.	Inspect the KV store e
11014	APIBulkChannelConnectionsFailed WARN	"Failed to get channel connections for bulk lookup" with <code>channel_id</code> . The bulk channel connection lookup used by the webapp channel indicator failed for one channel in the batch.	Single failures only aff are safe to ignore. Rep many channels mean t The rest of the batch i
11015	APITeamRewriteSet INFO	"Team rewrite set" with <code>team_id</code> , <code>connection</code> , <code>remote_team_name</code> , <code>user</code> . Audit log of a successful team name rewrite.	Informational and audi field is the admin who
11016	APITeamRewriteCleared INFO	"Team rewrite cleared" with <code>team_id</code> , <code>connection</code> , <code>user</code> . Audit log of a cleared rewrite.	Informational and audi

Emitted from plugin configuration load and validation.

Code	Name / Level	Log message and context	Troubleshooting
12000	ConfigSameConfigPassed WARN	"setConfiguration called with the existing configuration". The <code>OnConfigurationChange</code> hook fired but Mattermost passed the same config object that is already loaded.	Usually benign, fired during plugin activation and config reloads. Spammy occurrences suggest a tight loop calling <code>SavePluginConfig</code> . Check for misbehaving admin integrations.
12001	ConfigValidationWarn WARN	"Plugin configuration has validation warnings". Non-fatal validation warnings were raised while parsing plugin settings.	Read the warning text in the adjacent <code>warnings</code> field (connection with obsolete field, duplicate name, etc.). Edit the connection in System Console > Plugins > Cross Guard and save again to clear.

command.go (13000-13999)

Emitted from slash command handlers.

Code	Name / Level	Log message and context	Troubleshooting
13000	CommandOpenConnDialogFailed ERROR	"Failed to open connection dialog". The user ran a slash command (e.g., <code>/crossguard init-team</code>) without arguments and the plugin failed to open	Verify the Mattermost site URL is set correctly and that the server can reach its own plugin dialog endpoint. Check the <code>error</code> field for the specific RPC failure. Users can work around

Code	Name / Level	Log message and context	Troubleshooting
		the interactive selection dialog.	by passing the connection name directly on the command line.

service.go (14000-14999)

Emitted from the team and channel initialization service. This layer persists the team-to-connection and channel-to-connection links in the KV store, writes init and teardown announcement posts, and enforces header prefixes on linked channels.

Code	Name / Level	Log message and context	Troubleshooting
14000	ServiceInitTeamGetConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . <code>init-team</code> could not read the team's existing connection list before adding a new one.	Inspect the KV store. Usually a store outage with newly restored data without the plugin that <code>init-team</code> command is safe idempotent; retry at a later time if the team is healthy.
14001	ServiceAddTeamConnFailed ERROR	"Failed to add team connection" with <code>team_id</code> , <code>conn</code> . <code>init-team</code> failed to write the new team-to-connection link.	Inspect the KV store after the store recovery operation partially completed. If running <code>init-team</code> is safe.
14002	ServiceInitTeamReReadConnsFailed ERROR	"Failed to re-read team connections" with <code>team_id</code> . After	The write may still have succeeded. Run <code>/c status</code> in the team

Code	Name / Level	Log message and context	Troubleshooting
		writing the link, the service could not re-read the team's connections to confirm the update.	<code>/api/v1/teams/<</code> to verify.
14003	ServiceAddTeamInitializedFailed ERROR	"Failed to add team to initialized list" with <code>team_id</code> . Failed to update the "initialized teams" index used by global status.	Inspect the KV store. The primary link is still in global status may not be team. Re-running <code>python3</code> fixes the index.
14004	ServicePostTeamInitMsgFailed WARN	"Failed to post initialization message". The service successfully linked the team but failed to post the confirmation message in town-square.	Non-fatal. Confirm that a town-square character (default), and that the Guard bot has posted. Team initialization is complete.
14005	ServiceCheckTeamStatusFailed ERROR	"Failed to check team status" with <code>team_id</code> . Internal status lookup failed while the service was answering a status query.	Inspect the KV store. Affects <code>/crossguard</code> output.
14006	ServiceGetInitializedTeamsFailed ERROR	"Failed to get initialized teams". Global status	Inspect the KV store. Affects <code>GET /api/</code>

Code	Name / Level	Log message and context	Troubleshooting
		failed to load the list of initialized teams.	
14007	ServiceTeamStatusLookupTeamFailed WARN	"Failed to look up team for status" with <code>team_id</code> . Status endpoint failed to load a team record.	Team may have been removed since initialization. Run <code>teardown-team</code> to manually remove the team.
14008	ServiceTeamStatusGetConnsFailed WARN	"Failed to get team connections for status" with <code>team_id</code> . Status endpoint failed to load the team's connections.	Inspect the KV store for the team's connections.
14009	ServiceParseOutboundConnFailed WARN	"Failed to parse outbound connection configuration". A stored team link references an outbound connection that could not be parsed from the current plugin config.	The connection was deleted in System Console and is being linked. The link is orphaned: true in the output. Run <code>teardown-team</code> with the old name, or manually delete the connection in System Console.
14010	ServiceParseInboundConnFailed WARN	"Failed to parse inbound connection configuration". Same as 14009, but for inbound connections.	Same as 14009.

Code	Name / Level	Log message and context	Troubleshooting
14011	ServiceTeardownGetChanConnsFailed ERROR	"Failed to get channel connections" with <code>channel_id</code> . During team teardown, failed to list channel connections nested under the team.	Inspect the KV store. Orphaned channel IDs remain and can be cleaned up with explicit <code>teardown-channel</code> calls.
14012	ServiceTeardownGetTeamConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . Team teardown failed to load the team's own connection list.	Inspect the KV store.
14013	ServiceInitChanGetTeamConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . <code>init-channel</code> could not verify the parent team's connections.	Confirm the team is started (<code>/crossguard status</code>), not, run <code>init-teardown</code> .
14014	ServiceInitChanGetChanConnsFailed ERROR	"Failed to get channel connections" with <code>channel_id</code> . <code>init-channel</code> could not read the channel's existing connections.	Inspect the KV store.

Code	Name / Level	Log message and context	Troubleshooting
14015	ServiceAddChanConnFailed ERROR	"Failed to add channel connection" with <code>channel_id</code> , <code>conn</code> . Failed to write a new channel-to-connection link.	Inspect the KV store command is idempotent
14016	ServiceInitChanReReadConnsFailed ERROR	"Failed to re-read channel connections" with <code>channel_id</code> . After linking, re-read failed.	Verify with <code>/cross status</code> in the character
14017	ServiceChanHeaderPrefixFailed WARN	"Failed to update channel header with CrossGuard prefix" with <code>channel_id</code> . The channel was linked but the header could not be updated to add the Cross Guard marker.	Verify the bot has no channel permission target channel. The bot is still active; only the marker is missing.
14018	ServicePostChanInitMsgFailed WARN	"Failed to post channel init message". Channel was linked but the confirmation message could not be posted.	Confirm the bot can access target channel. Non
14019	ServiceRemoveChanGetConnsFailed ERROR	"Failed to get channel	Inspect the KV store

Code	Name / Level	Log message and context	Troubleshooting
		connections" with <code>channel_id</code> . <code>teardown-channel</code> could not load the current connections.	
14020	<code>ServiceRemoveChanConnFailed</code> ERROR	"Failed to remove channel connection" with <code>channel_id</code> , <code>conn</code> . Failed to delete a single channel-to-connection link.	Inspect the KV store retry. Orphaned link removed by re-running <code>teardown-channel</code>
14021	<code>ServiceRemoveChanReReadConnsFailed</code> ERROR	"Failed to re-read channel connections" with <code>channel_id</code> . Post-removal verify read failed.	Verify with <code>/cross status</code> .
14022	<code>ServiceDeleteChanConnsFailed</code> ERROR	"Failed to delete channel connections" with <code>channel_id</code> . Failed to delete the channel's full connection record during teardown.	Inspect the KV store retry teardown.
14023	<code>ServiceChanHeaderRemovePrefixFailed</code> WARN	"Failed to remove CrossGuard prefix from channel header" with <code>channel_id</code> . Could not strip the Cross Guard	Verify the bot has channel permission can edit the header remove the leftover

Code	Name / Level	Log message and context	Troubleshooting
		marker from the channel header during teardown.	
14024	ServicePostChanTeardownMsgFailed WARN	"Failed to post channel teardown message". Could not post the "channel unlinked" confirmation message.	Non-fatal. Confirm I post.
14025	ServiceTeardownTeamGetConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . Team teardown failed to load connections during cleanup.	Inspect the KV store
14026	ServiceRemoveTeamConnFailed ERROR	"Failed to remove team connection" with <code>team_id</code> , <code>conn</code> . Failed to delete a team-to-connection link during teardown.	Inspect the KV store retry.
14027	ServiceTeardownTeamReReadConnsFailed ERROR	"Failed to re-read team connections" with <code>team_id</code> . Post-removal verify read failed.	Verify with <code>/cross status</code> .
14028	ServiceRemoveTeamInitializedFailed ERROR	"Failed to remove team from initialized list" with <code>team_id</code> .	Inspect the KV store Global status may s team until re-run of

Code	Name / Level	Log message and context	Troubleshooting
		Failed to update the "initialized teams" index during teardown.	
14029	ServicePostTeamTeardownMsgFailed WARN	"Failed to post team teardown message". Non-fatal.	Confirm bot can post square.
14030	ServiceGlobalParseOutConnFailed WARN	"Failed to parse outbound connections". Global status could not parse an outbound connection from plugin config when answering GET /api/v1/status .	Review the outbound connection form in Console. The adjacent field names the specific failure.
14031	ServiceGlobalParseInConnFailed WARN	"Failed to parse inbound connections". Same as 14030 for inbound connections.	Same as 14030.
14032	ServiceMapParseOutConnFailed WARN	"Failed to parse outbound connections for map". Startup failed to build the internal outbound connection map.	Inspect the preceding error. The failing connection can be skipped from the map until fixed.
14033	ServiceMapParseInConnFailed WARN	"Failed to parse inbound connections for	Same as 14032.

Code	Name / Level	Log message and context	Troubleshooting
		map". Same as 14032 for inbound connections.	

inbound.go (15000-15999)

Emitted from the inbound dispatcher that consumes messages from the transport, applies idempotency, and materializes posts, edits, deletes, and reactions locally. Also covers the inbound file watcher loop.

Code	Name / Level	Log message and context	Troubleshooting
15000	InboundParseConnsFailed ERROR	"Failed to parse inbound connections". Startup failed to parse any inbound connections from plugin config.	Review all connection Console. F flagged in <code>error</code> file at least one p inbound cc messages received.
15001	InboundConnectFailed ERROR	"Failed to connect inbound" with <code>name</code> , <code>provider</code> . An inbound connection failed to dial its transport (NATS, Azure Queue, or Azure Blob).	Check prov codes in the log context 19xxx / 20x network re credentials subject/qu names. The loop will re configured

Code	Name / Level	Log message and context	Troubleshooting
15002	InboundSubscribeFailed ERROR	"Failed to subscribe inbound" with <code>name</code> . Connected to NATS but subscribe call failed.	Verify the inbound has subscription permission on the subject. Check the side authentication key.
15003	InboundSubscriptionEstablished INFO	"Inbound subscription established" with <code>name</code> , <code>provider</code> . Informational confirmation of a successful inbound subscription.	Informational message indicating a liveness check by the plugin rest client.
15004	InboundRelaySemaphoreFull WARN	"Relay semaphore full, dropping inbound message" with <code>conn</code> . The inbound concurrency semaphore was saturated; the current message was dropped.	Raise an alert. <code>RelayConcurrencyLimitReached</code> investigate slowness: <code>RelayCreatePostPayload</code> database lookup hooks on the Sustained the receive up.
15005	InboundUnmarshalFailed ERROR	"Failed to unmarshal inbound message" with <code>conn</code> . Received an envelope that parsed neither as JSON nor XML.	Check that the message uses a correct conversion and the <code>message_type</code> matches or the Capture the envelope for debugging.
15006	InboundPostMissingPayload ERROR	"Inbound post: missing payload" with <code>conn</code> . A <code>post</code> envelope arrived with no payload section.	Sender bug or corruption. Capture the message to confirm severity and file an issue.
15007	InboundUpdateMissingPayload ERROR	"Inbound update: missing payload" with <code>conn</code> . A	Same as 15006

Code	Name / Level	Log message and context	Troubleshooting
		<code>post_update</code> envelope arrived with no payload.	
15008	InboundDeleteMissingPayload ERROR	"Inbound delete: missing payload" with <code>conn</code> . A <code>post_delete</code> envelope arrived with no payload.	Same as 15007
15009	InboundReactionAddMissingPayload ERROR	"Inbound reaction add: missing payload" with <code>conn</code> . A <code>reaction_add</code> envelope arrived with no payload.	Same as 15007
15010	InboundReactionRemoveMissingPayload ERROR	"Inbound reaction remove: missing payload" with <code>conn</code> . A <code>reaction_remove</code> envelope arrived with no payload.	Same as 15007
15011	InboundTestMessageReceivedWithID INFO	"Received inbound test message" with <code>conn</code> , <code>id</code> . The inbound test subscriber received a test envelope with an ID.	Information correlates <code>APITestMessageReceived</code> (11004) on the test subscriber
15012	InboundTestMessageReceived INFO	"Received inbound test message" with <code>conn</code> . Test envelope without an ID.	Information correlates <code>APITestMessageReceived</code> (11004) on the test subscriber
15013	InboundUnknownMessageType WARN	"Unknown inbound message type" with <code>conn</code> , <code>type</code> . An envelope with an unrecognized <code>type</code> field arrived.	Usually the sender is too old or the receiver is too new. Check the plugin version
15014	InboundMissingMessageQueueFull ERROR	"Missing message: queue full, dropping message" with <code>conn</code> , <code>type</code> ,	Increase the parent queue settings, or

Code	Name / Level	Log message and context	Troubleshooting
		<code>remote_post_id</code> , <code>queue_size</code> . The retry queue for messages whose parent post was missing locally was full; the new message was dropped.	why parent arriving. See retry_dispatcher how the queue processes persistent messages usually message sender never the parent
15015	InboundMissingMessageQueued WARN	"Missing message: queuing for retry" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>queue_size</code> . A message arrived whose parent post has not yet been received; it was queued for retry.	Normal during traffic or on delivery. The dispatcher (see 22000) Rising queue across many indicates on problems in transport.
15016	InboundPostResolveFailed WARN	"Inbound post: resolve failed" with <code>conn</code> . Failed to resolve the target team/channel for an inbound post.	Most common the remote does not match team and reconfigured <code>/crossgu</code> <code>rewrite-target</code> Slash Command ensure team match on k

Code	Name / Level	Log message and context	Troubleshooting
15017	InboundPostIdempotencyLookupFailed WARN	"Inbound post: idempotency lookup failed, processing anyway" with <code>conn</code> , <code>postID</code> . The idempotency cache lookup failed, so the plugin cannot guarantee this post was not already created; it proceeds anyway.	Inspect the error. Risk: duplicate post message is by the transaction.
15018	InboundPostResolveUserFailed ERROR	"Inbound post: resolve user failed" with <code>conn</code> , <code>username</code> . Failed to resolve the post author to a local or sync user.	Check <code>Username</code> setting and <code>sync-user</code> succeeded username. <code>17xxx</code> cc the underlying The post is
15019	InboundPostRootMappingLookupFailed WARN	"Inbound post: failed to look up root mapping" with <code>conn</code> , <code>remote_root_id</code> . A thread reply arrived but the root-post mapping lookup failed.	Inspect the error.
15020	InboundPostRootNotFoundStandalone WARN	"Inbound post: root not found after retries, creating standalone" with <code>conn</code> , <code>remote_root_id</code> . After the retry dispatcher gave up waiting for the parent, the plugin created the reply as a standalone post at the top level of the channel.	Indicates that never arrived may have failed, or it). Check the logs for the <code>remote_r</code> Consider the retry dispatcher so slow arrivals are

Code	Name / Level	Log message and context	Troubleshooting
15021	InboundPostCreateFailed ERROR	"Inbound post: create failed" with <code>conn</code> . Mattermost rejected the <code>CreatePost</code> call.	Common cause: message length over 4096 characters archived or bot lacks permission validation error. See the error .
15022	InboundPostStoreMappingFailed ERROR	"Inbound post: failed to store post mapping" with <code>conn</code> , <code>remote_id</code> , <code>local_id</code> . The post was created locally, but saving the remote-to-local mapping failed.	Inspect the error. Future deletes of <code>remote_id</code> not find the <code>local_id</code> . Manual cleanup needed; check if this recurs.
15023	InboundUpdateMappingLookupFailed ERROR	"Inbound update: failed to look up post mapping" with <code>conn</code> , <code>remote_id</code> . Could not find the local post ID for an incoming edit.	Possible cause: original post reached threshold (check 15026) the mapping (15027-15028) is dropped.
15024	InboundUpdateGetLocalPostFailed ERROR	"Inbound update: failed to get local post" with <code>conn</code> , <code>local_id</code> . Mapping existed but Mattermost could not load the local post.	Post may have been deleted locally. Ignore since mapping exists.
15025	InboundUpdatePostFailed ERROR	"Inbound update: failed to update post" with <code>conn</code> , <code>local_id</code> . Mattermost rejected the <code>UpdatePost</code> call.	Common cause: message deleted or validation error. Check if bot lacks edit permission. Inspect the <code>update_post</code> field.

Code	Name / Level	Log message and context	Troubleshooting
15026	InboundDeleteMappingLookupFailed ERROR	"Inbound delete: failed to look up post mapping" with <code>conn</code> , <code>remote_id</code> . Could not find the local post ID for an incoming delete.	Same as 15025
15027	InboundDeleteSetFlagFailed ERROR	"Inbound delete: failed to set delete flag" with <code>conn</code> , <code>local_id</code> . Failed to set the <code>crossguard_delete_flag</code> that prevents the delete from echoing back through the outbound hook.	Inspect the error. With the delete back to the creating a for 10007 c side.
15028	InboundDeletePostFailed ERROR	"Inbound delete: failed to delete post" with <code>conn</code> , <code>local_id</code> . Mattermost rejected the <code>DeletePost</code> call.	Post may a deleted. In <code>error</code> fie
15029	InboundDeleteRemoveFlagFailed WARN	"Inbound delete: failed to remove delete flag" with <code>conn</code> , <code>local_id</code> . Failed to clear the delete flag after a successful delete.	Non-fatal. TTL and ex automatica
15030	InboundDeleteRemoveMappingFailed WARN	"Inbound delete: failed to remove post mapping" with <code>conn</code> , <code>remote_id</code> . Leaves a stale mapping entry.	Inspect the error. Stale only a spac does not a correctness
15031	InboundReactionMappingLookupFailed ERROR	"Inbound reaction: failed to look up post mapping" with <code>conn</code> , <code>remote_id</code> . Could not find the local post for a reaction event.	Same as 15025

Code	Name / Level	Log message and context	Troubleshooting
15032	InboundReactionResolveFailed WARN	"Inbound reaction: resolve failed" with <code>conn</code> . Failed to resolve the target channel for a reaction.	Same as 15031
15033	InboundReactionResolveUserFailed ERROR	"Inbound reaction: resolve user failed" with <code>conn</code> , <code>username</code> . Failed to resolve the reacting user.	Same as 15031
15034	InboundReactionAddFailed ERROR	"Inbound reaction: add failed" with <code>conn</code> , <code>post_id</code> . Mattermost rejected the <code>AddReaction</code> call.	Post may not exist, or the user may not have permission. Check the <code>error</code> field.
15035	InboundReactionRemoveFailed ERROR	"Inbound reaction: remove failed" with <code>conn</code> , <code>post_id</code> . Mattermost rejected the <code>RemoveReaction</code> call.	Same as 15031
15036	InboundFileWatcherStarted INFO	"File watcher started" with <code>conn</code> . The inbound file watcher goroutine started.	Informational. Shows up at plugin start for each inbound connector. Transfer encoding is not supported.
15037	InboundFileWatcherExited ERROR	"File watcher exited with error" with <code>conn</code> . The watcher loop exited, usually due to a fatal provider error.	Check the <code>error</code> field. It is correlated with 18xxx / 18xxx codes. The connector restarted on the next poll tick. Plugins should indicate connectivity or network outages.
15038	InboundFileMissingHeaders WARN	"Inbound file: missing required headers, skipping"	Something went wrong into the Connector.

Code	Name / Level	Log message and context	Troubleshooting
		with <code>key</code> . A staged file blob lacked the expected Cross Guard metadata headers.	container v envelope h Usually sta data. Capti by <code>key</code> fc
15039	InboundFileConnInactive WARN	"Inbound file: connection no longer active, skipping" with <code>conn</code> , <code>filename</code> . A file arrived for a connection that is no longer active on this instance.	Expected i after a <code>te</code> Persistent the sender relaying to connector
15040	InboundFileFilteredByPolicy INFO	"Inbound file filtered by policy" with <code>filename</code> , <code>conn</code> . A file was rejected by the connection's allow/deny extension filter.	Expected v are configu <code>file_fil</code> <code>file_fil</code> the connec filter was v
15041	InboundFileMappingLookupFailed ERROR	"Inbound file: post mapping lookup failed after retries" with <code>conn</code> , <code>remote_post_id</code> . Tried to find the local post ID for the file's parent post but the lookup kept failing.	Inspect the error. The 1 attached, a left in plac (Azure) or (NATS).
15042	InboundFileNoMappingFound WARN	"Inbound file: no post mapping found after retries" with <code>conn</code> , <code>remote_post_id</code> . The mapping lookup succeeded (no store error) but no local post exists for the remote ID.	The parent arrived or v out. Check for the <code>remote_p</code> Usually tra out-of-ord ID.
15043	InboundFileGetLocalPostFailed ERROR	"Inbound file: failed to get local post" with <code>local_post_id</code> . Mapping existed but Mattermost	Post may h deleted. Se

Code	Name / Level	Log message and context	Troubleshooting
		could not load the local post.	
15044	InboundFileUploadFailed ERROR	"Failed to upload file to Mattermost" with <code>filename</code> . Mattermost rejected the <code>UploadFile</code> call.	Check Mat <code>MaxFileS</code> storage qu available d the <code>error</code>
15045	InboundFileAttachFailed ERROR	"Failed to attach file to post" with <code>local_post_id</code> , <code>file_id</code> . File was uploaded but the association with the post failed.	Usually a p invalid po: Inspect the field.

connections.go (16000-16999)

Emitted from the outbound publisher pipeline: envelope serialization, multi-part splitting, provider publish with retries, and file upload to object store.

Code	Name / Level	Log message and context	Troubleshooting
16000	ConnectionsParseOutboundFailed ERROR	"Failed to parse outbound connections for relay". Failed to parse outbound connections at publish time.	Inspect the adjacent <code>error</code> field and fix the connection form in System Console. See also 14030.
16001	ConnectionsConnectOutboundFailed ERROR	"Failed to connect outbound for relay" with <code>name</code> ,	Verify provider reachability and credentials. Check correlated <code>18xxx</code> / <code>19xxx</code> / <code>20xxx</code>

Code	Name / Level	Log message and context	Troubleshooting
		<code>provider</code> . Outbound provider dial failed when a message needed to be published.	codes. Publish will retry per <code>PublishRetries</code> before giving up (16007).
16002	ConnectionsOutboundEstablished INFO	"Outbound connection established for relay" with <code>name</code> , <code>provider</code> . Informational confirmation.	Informational. Useful as a liveness signal.
16003	ConnectionsSerializeFailed ERROR	"Failed to serialize outbound message" with <code>name</code> , <code>format</code> . Could not serialize the envelope to JSON or XML.	Internal bug. Capture the adjacent <code>error</code> field and the source post ID for diagnosis.
16004	ConnectionsMessageSplit INFO	"Message split into parts for provider size limit" with <code>connection</code> , <code>parts</code> , <code>originalSize</code> . A large message was split into multiple transport parts.	Informational. Frequent splits with high <code>parts</code> counts indicate very large posts; consider whether file transfer would be more efficient

Code	Name / Level	Log message and context	Troubleshooting
16005	ConnectionsSerializePartFailed ERROR	"Failed to serialize message part" with <code>name</code> , <code>part</code> . Part of a split message failed to serialize.	Same as 16003.
16006	ConnectionsPublishPartFailed ERROR	"Failed to publish message part" with <code>name</code> , <code>part</code> . Publish of a single part failed on one attempt.	Transient if followed by successful retry. If all parts for a message fail see 16007.
16007	ConnectionsPublishAfterRetriesFail ERROR	"Failed to publish to outbound after retries" with <code>name</code> . All publish retries for a part were exhausted. The message was dropped.	Escalate: transport is persistently unavailable or rejecting messages. Check provider logs and credentials. Raising <code>PublishRetries</code> only helps if the failure is genuinely transient.
16008	ConnectionsGetFileInfoFailed ERROR	"Failed to get file info for relay" with <code>file_id</code> , <code>post_id</code> . FileInfo lookup failed before upload.	File may have been removed. Inspect the <code>error</code> field.
16009	ConnectionsSkipOversizedFile WARN	"Skipping oversized file for relay" with	Expected when limits are enforced. Raise the connection's max-file-

Code	Name / Level	Log message and context	Troubleshooting
		<code>file</code> , <code>size</code> , <code>max</code> . A file exceeded the per-connection max size and was skipped.	size if intentional. The post is still relayed without the attachment
16010	ConnectionsDownloadFileFailed ERROR	"Failed to download file for relay" with <code>file_id</code> , <code>file</code> . Failed to download the local file bytes from Mattermost before upload.	Check Mattermost file storage health (S3, local disk, etc).
16011	ConnectionsOutboundFileFiltered INFO	"Outbound file filtered by policy" with <code>file</code> , <code>conn</code> . File was filtered out by the outbound extension filter.	Expected when filters are configured. Adjust the outbound filter if unintentional.
16012	ConnectionsFileSemaphoreFull WARN	"File semaphore full, skipping file upload" with <code>file</code> , <code>conn</code> . The file transfer concurrency semaphore was full.	Raise the file transfer concurrency setting, or investigate object store slowness. The post is still relayed without this attachment.
16013	ConnectionsUploadFileFailed ERROR	"Failed to upload file" with <code>key</code> , <code>conn</code> . Upload to JetStream Object Store or	Check provider logs and credentials. The referenced file will not arrive on the receiving

Code	Name / Level	Log message and context	Troubleshooting
		Azure Blob failed.	side; the post envelope still goes through.

sync_user.go (17000-17999)

Emitted from the synthetic relay user subsystem, which creates local Mattermost accounts to represent remote authors when `UsernameLookup` does not find a match.

Code	Name / Level	Log message and context	Troubleshooting
17000	SyncUserTruncatedUsername WARN	"Truncated sync username to fit limit" with <code>original</code> , <code>munged</code> . A remote username exceeded the Mattermost 64-char username limit and was shortened.	Normal for very long remote usernames. The sync user is still created under the truncated name. Users with the same truncated prefix will collide; consider naming conventions if this happens often.
17001	SyncUserLookupFallback DEBUG	"Username lookup did not find local user, falling back to sync user" with <code>username</code> , <code>conn</code> . The <code>UsernameLookup</code> setting is enabled but no matching local user was found.	Expected when the remote username is not a local user. A sync user is created and used instead.
17002	SyncUserAddToTeamFailed WARN	"Failed to add sync user to team" with <code>user_id</code> , <code>team_id</code> . Failed to add the new	Check team membership limits and that the team allows the sync user's email domain. The post will

Code	Name / Level	Log message and context	Troubleshooting
		sync user to the target team.	still attempt to proceed but may not appear until the user is a team member.
17003	SyncUserAddToChannelFailed WARN	"Failed to add sync user to channel" with <code>user_id</code> , <code>channel_id</code> . Failed to add the sync user to the target channel.	Check channel permissions and whether the channel is private or restricted. The post may not be created until the user is a channel member.

azure_blob_provider.go (18000-18999)

Emitted from the Azure Blob WAL provider. This provider batches outgoing envelopes into a write-ahead log on local disk, uploads them as blobs, stages deferred file attachments via a companion file, and uses lease-based locking so only one receiver per container processes a given segment. Most codes relate to WAL rotation, recovery, or lock contention.

Code	Name / Level	Log message and context	Troubleshooting
18000	AzureBlobContainerCreateRetry WARN	"Azure Blob: transient container create error, retrying" with <code>container</code> , <code>attempt</code> , <code>delay</code> . Container creation was retried on startup.	Occasional retries expected. Persiste indicate the creder cannot create cont (permissions issue Storage is unreach Check the <code>error</code> escalate if attempt

Code	Name / Level	Log message and context	Troubleshooting
18001	AzureBlobWALInTempStorage WARN	"Azure Blob provider: WAL directory is in temp storage and may not survive container restarts" with <code>wal_dir</code> . The WAL directory resolved to <code>\$TMPDIR</code> because no persistent path is configured.	Important in production a persistent <code>WALDIR</code> in System Console container restarts are still in the local are lost permanent acceptable in dev external durability.
18002	AzureBlobWALDirFsyncFailed WARN	"Azure Blob: WAL dir fsync failed" with <code>dir</code> . Fsync of the WAL directory failed.	Check filesystem h space, and inode e Fsync failure mean is not guaranteed f previous write.
18003	AzureBlobMarshalPendingFailed WARN	"Azure Blob: failed to marshal pending files". Could not serialize pending file refs when preparing a WAL flush.	Internal bug. Captu payload and file an
18004	AzureBlobWriteCompanionFailed ERROR	"Azure Blob: failed to write companion files.json" with <code>path</code> . The companion file tracks deferred	Check disk space ; permissions on the directory. Without companion, file att for this segment m

Code	Name / Level	Log message and context	Troubleshooting
		file uploads for a WAL segment; writing it failed.	
18005	AzureBlobShutdownLeftWALFiles WARN	"Azure Blob: shutdown left WAL files for recovery" with <code>count</code> , <code>wal_dir</code> . At shutdown, this many WAL files had not yet been uploaded.	Expected during fast shutdown. Recover on next startup and upload leftover files (watch 18043). If the WAL is ephemeral (see 18043), files are lost.
18006	AzureBlobWALFsyncRotationFailed WARN	"Azure Blob: WAL fsync failed during rotation" with <code>path</code> . Fsync during WAL segment rotation failed.	Check disk health. Recovery may still upload but is not guaranteed.
18007	AzureBlobWALCloseRotationFailed ERROR	"Azure Blob: WAL close failed during rotation, skipping upload" with <code>path</code> . The close-on-rotate failed; the segment is skipped for this rotation cycle.	Inspect the filesystem. Segment is left on disk; the next recovery attempt will skip it.
18008	AzureBlobWALUploadFailed ERROR	"Azure Blob: WAL upload failed, leaving for recovery" with <code>path</code> . Uploading a	Verify credentials, existence, and network. Segment stays on disk; will be retried by the recovery process on the next tick.

Code	Name / Level	Log message and context	Troubleshooting
		rotated WAL segment to Azure Blob failed.	
18009	AzureBlobDeleteWALFailed WARN	"Azure Blob: failed to delete WAL after upload" with path . Segment uploaded successfully but local delete failed.	Check filesystem permissions. Next recovery will retry
18010	AzureBlobMarshalFailedRefsFailed ERROR	"Azure Blob: failed to marshal failed refs for companion rewrite" with path , count . Internal marshal error during companion rewrite.	Internal bug. Capture path and count for
18011	AzureBlobRewriteCompanionFailed ERROR	"Azure Blob: failed to rewrite companion files.json with failed refs" with path , count . Could not rewrite the companion with the remaining failed refs after a partial upload.	Check filesystem for Failed refs may be with a stale view of

Code	Name / Level	Log message and context	Troubleshooting
18012	AzureBlobDeleteCompanionFailed WARN	"Azure Blob: failed to delete companion files.json" with <code>path</code> . All deferred uploads completed but deleting the companion failed.	Check filesystem permissions. Next pass will clean up l companion files.
18013	AzureBlobDeferredFileFetchFailed ERROR	"Azure Blob: deferred file fetch failed" with <code>file_id</code> , <code>post_id</code> . Failed to re-read a local Mattermost file for deferred upload.	File may have beer from Mattermost a envelope was writt the error.
18014	AzureBlobDeferredFileUploadFailed ERROR	"Azure Blob: deferred file upload failed" with <code>file_id</code> , <code>post_id</code> . Failed to upload a deferred file attachment to the Azure Blob container.	Check credentials network. The file is the next tick via th refs list.
18015	AzureBlobShutdownMarshalResidualFailed ERROR	"Azure Blob: shutdown: failed to marshal residual pending files" with	Internal error. Resi is lost; next recove the WAL dir but m deferred file refs.

Code	Name / Level	Log message and context	Troubleshooting
		<code>count</code> . At shutdown, failed to serialize residual pending state.	
18016	AzureBlobShutdownPersistResidualFailed ERROR	"Azure Blob: shutdown: failed to persist residual pending files" with <code>count</code> , <code>path</code> . At shutdown, failed to write the residual state file to disk.	Check disk space. state is lost.
18017	AzureBlobShutdownPersistedResidual WARN	"Azure Blob: shutdown persisted residual pending files for recovery" with <code>count</code> , <code>path</code> . Residual pending state was successfully persisted for next startup.	Informational, expected during clean shutdown in-flight files.
18018	AzureBlobListFailed ERROR	"Azure Blob: list failed" with <code>container</code> . The receiver could not list blobs in the WAL container.	Check network, credentials, and container existence. Persistent failures inbound WAL processing.

Code	Name / Level	Log message and context	Troubleshooting
18019	AzureBlobDeleteRetryFailed WARN	"Azure Blob: delete retry failed (marker present)" with blob . Retry of a blob delete (after processed-marker write) failed.	Blob will be retried next poll. The proc marker prevents reprocessing.
18020	AzureBlobDownloadFailed WARN	"Azure Blob: download failed" with blob . Failed to download a WAL segment blob during receiver poll.	Check network. Se retried on the next
18021	AzureBlobHandlerError WARN	"Azure Blob: handler error, will retry blob" with blob . The downstream inbound handler returned an error for this segment.	Correlate with the 15xxx error (usu InboundPostCrea or similar). The blo until max attempts
18022	AzureBlobDeleteFailed WARN	"Azure Blob: delete failed" with blob . Failed to delete a WAL segment after successful processing.	Check credentials. marker prevents d processing even if repeatedly.

Code	Name / Level	Log message and context	Troubleshooting
18023	AzureBlobWriteProcessedMarkerFailed WARN	"Azure Blob: failed to write processed marker" with blob . Could not write the "processed" marker that prevents reprocessing.	Check credentials. the marker, the seq be reprocessed on poll. Handlers are i so this is survivabl wastes work.
18024	AzureBlobClearProcessedMarkerFailed WARN	"Azure Blob: failed to clear processed marker" with blob . Cleanup of a stale processed marker failed.	Non-fatal. Markers TTL and are clean next recovery pass
18025	AzureBlobLockGetFailed WARN	"Azure Blob: lock get failed" with blob . Failed to read the receiver lock blob.	Check credentials the lock blob path Transient failures a expected; persiste prevent any receiv acquiring the lock.
18026	AzureBlobLockTokenGenFailed WARN	"Azure Blob: lock token generation failed" with blob . Failed to generate a random lock token (crypto rand failure).	Indicates a broken Escalate immediat
18027	AzureBlobLockSetFailed WARN	"Azure Blob: lock set failed"	Either another rece currently holds the

Code	Name / Level	Log message and context	Troubleshooting
		with blob . Failed to write the receiver lock blob.	(normal in multi-nc credentials are bro is the only instance persists, inspect th
18028	AzureBlobCorruptLockReclaimFailed WARN	"Azure Blob: corrupt lock reclaim failed" with blob . Tried to reclaim a corrupt (unparseable) lock blob.	Manually inspect a the lock blob in the This is a rare state a partial write.
18029	AzureBlobStaleLockReclaimFailed WARN	"Azure Blob: stale lock reclaim failed" with blob . Tried to reclaim a stale lock (owner died without releasing) and failed to overwrite.	Another node may reclaimed first. Wa cycle and check ag
18030	AzureBlobLockReleaseFailed WARN	"Azure Blob: lock release failed" with blob . Failed to release the receiver lock during shutdown.	Non-fatal at shutdo lock reclaim in ano instance will recov lease expiry.
18031	AzureBlobLockReleaseGetFailed WARN	"Azure Blob: lock release get failed" with blob . Failed to read the lock	Same as 18030.

Code	Name / Level	Log message and context	Troubleshooting
		blob during release to verify ownership.	
18032	AzureBlobLockReleaseCorrupt WARN	"Azure Blob: lock release saw corrupt value, leaving" with <code>blob</code> . At release, the lock blob contents were corrupt and were left for reclaim.	Non-fatal. Investigate tampering or crash. The next reclaim pass handles it.
18033	AzureBlobLockReleaseReclaimedByOther WARN	"Azure Blob: skipping release, lock reclaimed by another node" with <code>blob</code> , <code>current_node</code> . Another instance reclaimed the lock before this one released it.	Usually means this lease expired while processing. Check skew between node lease duration setting whether the current too long.
18034	AzureBlobLockReleaseFailed2 WARN	"Azure Blob: lock release failed" with <code>blob</code> . Second-stage lock release (delete of the lock blob) failed.	Same as 18030.

Code	Name / Level	Log message and context	Troubleshooting
18035	AzureBlobWALRecoveryScanRootFailed WARN	"Azure Blob: WAL recovery: failed to scan root" with <code>root</code> . Startup recovery failed to scan the root of the WAL directory.	Check WAL directory and permissions. If recovery will run until fixed.
18036	AzureBlobWALRecoveryScanDirFailed WARN	"Azure Blob: WAL recovery: failed to scan directory" with <code>dir</code> . Failed to scan a WAL subdirectory during recovery.	Same as 18035.
18037	AzureBlobWALRecoverySkipUnrecognized WARN	"Azure Blob: WAL recovery: skipping unrecognized file" with <code>path</code> . Recovery skipped a file with an unknown name pattern.	Inspect the path. If debug data, delete legitimate file, Crofs does not recognize format.
18038	AzureBlobWALRecoveryUploadFailed ERROR	"Azure Blob: WAL recovery upload failed" with <code>path</code> . Recovery failed to upload a leftover WAL segment.	Check credentials network. Segment disk for the next reattempt.

Code	Name / Level	Log message and context	Troubleshooting
18039	AzureBlobWALRecoveryDeleteFailed WARN	"Azure Blob: WAL recovery delete failed" with <code>path</code> . After recovery upload, the local segment delete failed.	Check filesystem permissions.
18040	AzureBlobWALRecoveryReadCompanionFail WARN	"Azure Blob: WAL recovery: failed to read companion file" with <code>path</code> . Recovery failed to read a segment's companion file.	Check filesystem permissions. Recovery will skip segment's deferred recovery.
18041	AzureBlobWALRecoveryMalformedCompanion WARN	"Azure Blob: WAL recovery: malformed companion file" with <code>path</code> . A companion file exists but is unparseable.	Move the companion file and investigate. The recovery is retried from scratch.
18042	AzureBlobWALRecoveryMarshalRemaining WARN	"Azure Blob: WAL recovery: failed to marshal remaining refs" with <code>path</code> , <code>count</code> . Internal error during recovery companion rewrite.	Internal bug.

Code	Name / Level	Log message and context	Troubleshooting
18043	AzureBlobWALRecoveryRewriteCompanion WARN	"Azure Blob: WAL recovery: failed to rewrite companion file" with <code>path</code> , <code>count</code> . Could not rewrite a recovered companion file.	Check filesystem permissions.
18044	AzureBlobFileListFailed ERROR	"Azure Blob: file list failed" with <code>container</code> . Failed to list deferred file blobs during receiver polling.	Check credentials network. Persisten stop inbound file d post envelopes ma
18045	AzureBlobFileDeleteRetryFailed WARN	"Azure Blob: file delete retry failed (marker present)" with <code>blob</code> . Retry of deferred file blob delete failed.	Blob will be retried
18046	AzureBlobFileDownloadFailed WARN	"Azure Blob: file download failed" with <code>blob</code> . Failed to download a deferred file blob.	Check network. Re next poll.
18047	AzureBlobFileHandlerError WARN	"Azure Blob: file handler error" with <code>blob</code> . The file handler	Correlate with the <code>15041 - 15045</code> ir error.

Code	Name / Level	Log message and context	Troubleshooting
		rejected a downloaded deferred file.	
18048	AzureBlobFileDeleteFailed WARN	"Azure Blob: file delete failed" with <code>blob</code> . Failed to delete a deferred file blob after successful processing.	Check credentials.

azure_provider.go (19000-19999)

Emitted from the Azure Queue provider, which uses an Azure Storage Queue for envelopes and a paired Azure Blob container for file attachments.

Code	Name / Level	Log message and context	Troubleshooting
19000	AzureQueueCreateQueueFailed WARN	"Azure Queue: could not create queue (may already exist)" with <code>queue</code> . Idempotent startup create returned an error.	Usually benign (queue already exists). If persistent, the credentials cannot list/create queues. Verify account permissions.
19001	AzureQueueCreateContainerFail WARN	"Azure Blob: could not create container (may already exist)" with <code>container</code> . Idempotent blob container create returned an error.	Same as 19000.

Code	Name / Level	Log message and context	Troubleshooting
19002	AzureQueueDequeueFailed ERROR	"Azure Queue dequeue failed" with queue . Failed to dequeue a batch of messages.	Check network and credentials. Persistent failures stop inbound traffic on this queue. The receiver loop retries on the configured interval.
19003	AzureQueueDecodeFailed ERROR	"Azure Queue: failed to decode message" with queue . A dequeued message body was not a valid Cross Guard envelope.	Capture the raw body. Likely a sender producing malformed data, or an unrelated process writing to the queue.
19004	AzureQueueHandlerRetry WARN	"Azure Queue: handler returned error, message will retry" with queue . The downstream inbound handler returned an error; the message is left on the queue for the next visibility timeout.	Correlate with the adjacent 15xxx inbound error. Azure Storage Queue's poison-message threshold (dequeue count) will eventually drop the message if it keeps failing.
19005	AzureQueueDeleteProcessedFail WARN	"Azure Queue: failed to delete processed message" with queue . Message was processed but the delete call failed.	Message will be re-delivered at lock expiry; inbound handlers are idempotent via the idempotency cache, so duplicates are suppressed.
19006	AzureQueueBlobListFailed ERROR	"Azure Blob list failed" with	Check credentials and network.

Code	Name / Level	Log message and context	Troubleshooting
		<code>container</code> . Failed to list deferred file blobs in the paired container.	
19007	AzureQueueBlobDownloadFailed WARN	"Azure Blob: failed to download" with <code>blob</code> . Failed to download a deferred file blob.	Check network. Retried on the next poll.
19008	AzureQueueBlobHandlerError WARN	"Azure Blob: handler returned error" with <code>blob</code> . File handler rejected a downloaded blob.	Correlate with the adjacent <code>15041</code> - <code>15045</code> inbound file error.
19009	AzureQueueBlobDeleteFailed WARN	"Azure Blob: failed to delete after processing" with <code>blob</code> . Failed to delete a blob after successful processing.	Check credentials.

nats_provider.go (20000-20999)

Emitted from the NATS transport provider and its JetStream Object Store file handling.

Code	Name / Level	Log message and context	Troubleshooting
20000	NATSDownloadFileFailed ERROR	"Failed to download file from object store" with <code>key</code> . JetStream Object Store download failed.	Check JetStream health on the NATS broker (<code>nats stream ls</code>), confirm the object store bucket exists, and check the <code>key</code> has not been pruned.

Code	Name / Level	Log message and context	Troubleshooting
20001	NATSFileHandlerError WARN	"File handler returned error" with <code>key</code> . Downstream file handler rejected a downloaded object.	Correlate with the adjacent <code>15041 - 15045</code> inbound file error.
20002	NATSDisconnected WARN	NATS client disconnect callback with <code>name</code> . Log message is generated by the NATS client library.	Transient if followed by 20003 soon after. Persistent disconnects mean the broker is unreachable or the client is being forcibly kicked (auth rotation, account cap).
20003	NATSReconnected INFO	NATS client reconnect callback with <code>name</code> . Informational.	Informational, confirms recovery after a 20002 event.

prompt.go (21000-21999)

Emitted from the inbound prompt subsystem. Prompts are interactive posts (Accept / Block buttons) shown when a new inbound connection tries to deliver to a team or channel that has not yet opted in.

Code	Name / Level	Log message and context	Troubleshooting
21000	PromptGetConnPromptFailed ERROR	"Failed to get connection prompt" with <code>team_id</code> , <code>conn</code> . Failed to read the stored team-level prompt state.	Inspect the KV store error.
21001	PromptGetTownSquareFailed ERROR	"Failed to get town-square for prompt"	Verify the team has a town-square channel.

Code	Name / Level	Log message and context	Troubleshooting
		with <code>team_id</code> . Failed to find town-square when posting a team-level prompt.	Rare; only occurs if the default channel was deleted, which Mattermost normally prevents.
21002	PromptCreatePostFailed ERROR	"Failed to create connection prompt post" with <code>team_id</code> , <code>conn</code> . Failed to post the prompt message.	Confirm the bot can post in town-square and the post payload does not violate server limits.
21003	PromptSavePromptFailed ERROR	"Failed to save connection prompt" with <code>team_id</code> , <code>conn</code> . Failed to save the prompt state after posting.	Inspect the KV store error.
21004	PromptAcceptGetPromptFailed ERROR	"Failed to get connection prompt" with <code>team_id</code> , <code>conn</code> . Accept handler could not load the stored prompt state.	Inspect the KV store error. The admin may need to click Accept again after the store recovers.
21005	PromptDeletePromptFailed ERROR	"Failed to delete connection prompt" with <code>team_id</code> , <code>conn</code> . Failed to delete the prompt state after Accept.	Prompt may reappear on the next inbound message until the delete succeeds. Use <code>reset-prompt</code> to manually clear.

Code	Name / Level	Log message and context	Troubleshooting
21006	PromptBlockGetPromptFailed ERROR	"Failed to get connection prompt" with <code>team_id</code> , <code>conn</code> . Block handler could not load state.	Inspect the KV store error.
21007	PromptSetBlockedFailed ERROR	"Failed to update connection prompt to blocked" with <code>team_id</code> , <code>conn</code> . Failed to mark the prompt as blocked.	Inspect the KV store error.
21008	PromptGetChanPromptFailed ERROR	"Failed to get channel connection prompt" with <code>channel_id</code> , <code>conn</code> . Failed to read channel-level prompt state.	Inspect the KV store error.
21009	PromptCreateChanPostFailed ERROR	"Failed to create channel connection prompt post" with <code>channel_id</code> , <code>conn</code> . Failed to post the channel-level prompt.	Confirm the bot can post in the target channel.
21010	PromptSaveChanPromptFailed ERROR	"Failed to save channel connection prompt" with <code>channel_id</code> , <code>conn</code> . Failed to save channel-level prompt state.	Inspect the KV store error.
21011	PromptChanAcceptGetFailed ERROR	"Failed to get channel connection	Inspect the KV store error.

Code	Name / Level	Log message and context	Troubleshooting
		prompt" with channel_id , conn . Channel-level Accept handler failed to load state.	
21012	PromptDeleteChanPromptFailed ERROR	"Failed to delete channel connection prompt" with channel_id , conn . Failed to delete channel-level prompt state after Accept.	Use reset-channel- prompt to clear manually.
21013	PromptChanBlockGetFailed ERROR	"Failed to get channel connection prompt" with channel_id , conn . Channel-level Block handler failed to load state.	Inspect the KV store error.
21014	PromptSetChanBlockedFailed ERROR	"Failed to update channel connection prompt to blocked" with channel_id , conn . Failed to set channel-level blocked state.	Inspect the KV store error.
21015	PromptGetPostForUpdateFailed ERROR	"Failed to get prompt post for update" with post_id . Could not load the prompt post to update its displayed message after user action.	Post may have been deleted. Non-fatal; the state is still updated.

Code	Name / Level	Log message and context	Troubleshooting
21016	PromptUpdatePostFailed ERROR	"Failed to update prompt post" with <code>post_id</code> . Could not update the prompt post to reflect the user's Accept/Block action.	Check bot post permissions. Non-fatal; the prompt state change is persisted even if the visible post still shows the old buttons.

retry_dispatch.go (22000-22999)

Emitted from the retry queue that holds inbound messages whose parent post has not yet been received. A background ticker reprocesses the queue.

Code	Name / Level	Log message and context	Troubleshooting
22000	RetryDispatchSucceeded WARN	"Missing message: retry succeeded" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>attempt</code> . A queued message was successfully dispatched after its parent arrived.	Informational for operators but logged at WARN so it is visible. Rising <code>attempt</code> counts indicate systemic out-of-order delivery.
22001	RetryDispatchStillMissing WARN	"Missing message: still missing after retry" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>attempt</code> , <code>next_retry_in</code> . Parent is still missing; the message stays queued.	Expected for a few attempts during out-of-order delivery. If this repeats past <code>attempt</code> = max-retries, see 22004.

Code	Name / Level	Log message and context	Troubleshooting
22002	RetryDispatchDropMaxAge ERROR	"Missing message: dropped, exceeded max age" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>age</code> . Parent never arrived within max age; message dropped.	The parent post never made it through. Check the sender's logs for the <code>remote_post_id</code> and see whether an earlier error (15xxx, 16xxx, or sender-side) blocked it. Consider raising the max-age setting if senders are legitimately slow.
22003	RetryDispatchDropUnmarshal ERROR	"Missing message: dropped, payload unmarshal failed on retry" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> . A queued message failed to unmarshal on retry.	Usually indicates corruption at rest or a codec version mismatch. The envelope was valid when first received, so investigate memory or KV store corruption.
22004	RetryDispatchDropMaxRetries ERROR	"Missing message: dropped after max retries" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>attempts</code> . A queued message exceeded the max retry count.	Parent cannot be resolved. Investigate whether the parent post is being filtered out (e.g., blocked prompt, inactive channel, unresolved user) rather than delayed.

store/caching.go (23000-23999)

Emitted from the in-memory cache that sits in front of the KV store on each node.

Code	Name / Level	Log message and context	Troubleshooting
23000	StoreCachePublishInvalidationFailed WARN	"Failed to publish cache invalidation event" with <code>event</code> , <code>key</code> . Could not publish a cluster cache invalidation message after a KV write.	In a multi-node Mattermost cluster, other nodes may serve stale data until the next TTL (default 30 seconds). Check that the Mattermost cluster plugin message bus is healthy. Single-node deployments can ignore this code.

request.go (24000-24999)

Emitted from the team admin connection request workflow. When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can submit link requests that System Admins approve or deny via interactive DM buttons. These codes cover request creation, approval, denial, cancellation, and notification delivery.

Code	Name / Level	Log message and context	Troubleshooting
24000	RequestGetFailed ERROR	"Failed to get connection request" with <code>team_id</code> , <code>conn_key</code> . Also used during cancellation (WARN level): "Failed to get connection request for cancellation". KV store lookup for an existing pending request failed.	Check the <code>error</code> field for the underlying KV store error. Usually transient. If repeated, check KV store health. The request creation or cancellation will be aborted.

Code	Name / Level	Log message and context	Troubleshooti
24002	RequestGetDMChannelFailed ERROR	"Failed to get DM channel for connection request" with <code>admin_id</code> . Could not open a DM channel between the bot and a System Admin to deliver the request notification.	Verify the bot user exists and the admin account is active. This admin will not receive the request DM, but others still will. If all admins fail, the request may be saved with notification posts.
24003	RequestCreatedMPostFailed ERROR	"Failed to create DM post for connection request" with <code>admin_id</code> . The DM channel was opened but the post with Approve/Deny buttons could not be created.	Check Mattermost server health and post creation permissions. This admin will not see the request. If all admins fail, the request will be saved but no one can act on it from the UI.
24004	RequestSaveFailed ERROR	"Failed to save connection request" with <code>team_id</code> , <code>conn_key</code> . Could not persist the pending request to the KV store after DM posts were created.	The DM posts will be cleaned up automatically. Inspect the KV store error. The Team Admin will see a generic "failed to save message. Retry the command."

Code	Name / Level	Log message and context	Troubleshooti
24005	RequestApproveGetFailed ERROR	"Failed to get connection request for approve" with <code>team_id</code> , <code>conn_key</code> . When a System Admin clicks Approve, the handler could not load the pending request from KV store.	Check KV stor health. The admin will see "Failed to chec request status in the post action respons The request remains pendin and can be retried.
24006	RequestApproveExecFailed ERROR	"Failed to execute approved connection request" with <code>team_id</code> , <code>conn_key</code> . The request was approved but the underlying <code>initTeamForCrossGuard</code> call failed.	Inspect the nested error message. Common causes: team deleted, connection config change KV store error. The request remains pending. Fix th underlying issi and have the admin click Approve again
24007	RequestApproveDeleteFailed ERROR	"Failed to delete approved connection request" with <code>team_id</code> , <code>conn_key</code> . The connection was linked successfully but the pending request record could not be cleaned up.	Non-critical: th connection is active. The sta request recorc will be detecte as "no longer active" if someone click Approve/Deny again. No activ

Code	Name / Level	Log message and context	Troubleshooti
			needed unless persists.
24008	RequestApproveNotifyFailed ERROR	"Failed to get DM channel for requester notification" or "Failed to notify requester" with <code>requester_id</code> . Could not send the approval/denial notification DM to the requesting Team Admin.	Check that the requester account still exists and the bot can open a DM. The connection action (approve/deny) was still completed successfully; only the notification was lost.
24009	RequestDenyDialogFailed ERROR	"Failed to open deny dialog" with <code>team_id</code> , <code>conn_key</code> . The Deny button was clicked but the interactive dialog (for entering a reason) could not be opened.	Check Mattermost server health. The request remains pending. The admin can try clicking Deny again or use the Approve button instead.
24010	RequestDenyGetFailed ERROR	"Failed to get connection request for deny submit" with <code>team_id</code> , <code>conn_key</code> . After the deny dialog was submitted, the handler could not load the pending request.	Check KV store health. The denial was not processed. The admin can retry from the original DM buttons.

Code	Name / Level	Log message and context	Troubleshooting
24011	RequestDenyDeleteFailed ERROR	"Failed to delete denied connection request" with <code>team_id</code> , <code>conn_key</code> . Also used during cancellation (WARN level): "Failed to delete connection request during cancellation". The deny or cancel succeeded logically but the KV record was not removed.	Non-critical: same as 24008. The stale record will be detected as "no longer active" on next interaction.
24012	RequestDenyNotifyFailed ERROR	Reserved for denial notification failures. Currently shares the <code>RequestApproveNotifyFailed</code> code path (24008) for requester notification.	See 24008. This code is allocated for future use. Deny notifications need a distinct code path.
24013	RequestUpdatePostFailed WARN	"Failed to get request post for update" or "Failed to update request post" with <code>post_id</code> . After approval, denial, or cancellation, the handler could not update one of the admin DM posts to remove the Approve/Deny buttons.	Non-critical: the request action was still processed. The admin may see stale buttons, but clicking them will show "This request is no longer active." Post may have been deleted manually.
24014	RequestNoSystemAdmins WARN	"No system admins available to review connection request" with <code>team_id</code> , <code>conn_key</code> . The Team Admin submitted a request but the system has no admin users to notify.	Ensure at least one System Admin account exists and is active. The request was not created. The

Code	Name / Level	Log message and context	Troubleshooti
			Team Admin sees "no syste admins availab to review your request."
24015	RequestConnRemovedFromConfig WARN	"Connection removed from config while request was pending" with <code>team_id</code> , <code>conn_key</code> . A System Admin clicked Approve but the connection no longer exists in plugin configuration.	The request is automatically cancelled and the requester i notified. The connection mu be re-added to the configuration before a new request can be submitted.

[Back to Overview](#)